

DSA0409 – Fundamentals of Data Science

CO1 Lab Questions

Done By:

Name: N Abishek Kumaran

Reg no: 192573006

Department: CSE – DS

Date: 07.10.2026

Question 1: Student Subject Averages using NumPy

Program code:

```
import numpy as np

student_scores = np.array([
    [80, 75, 85, 70],
    [90, 85, 80, 75],
    [70, 80, 90, 85],
    [85, 90, 95, 80]
])

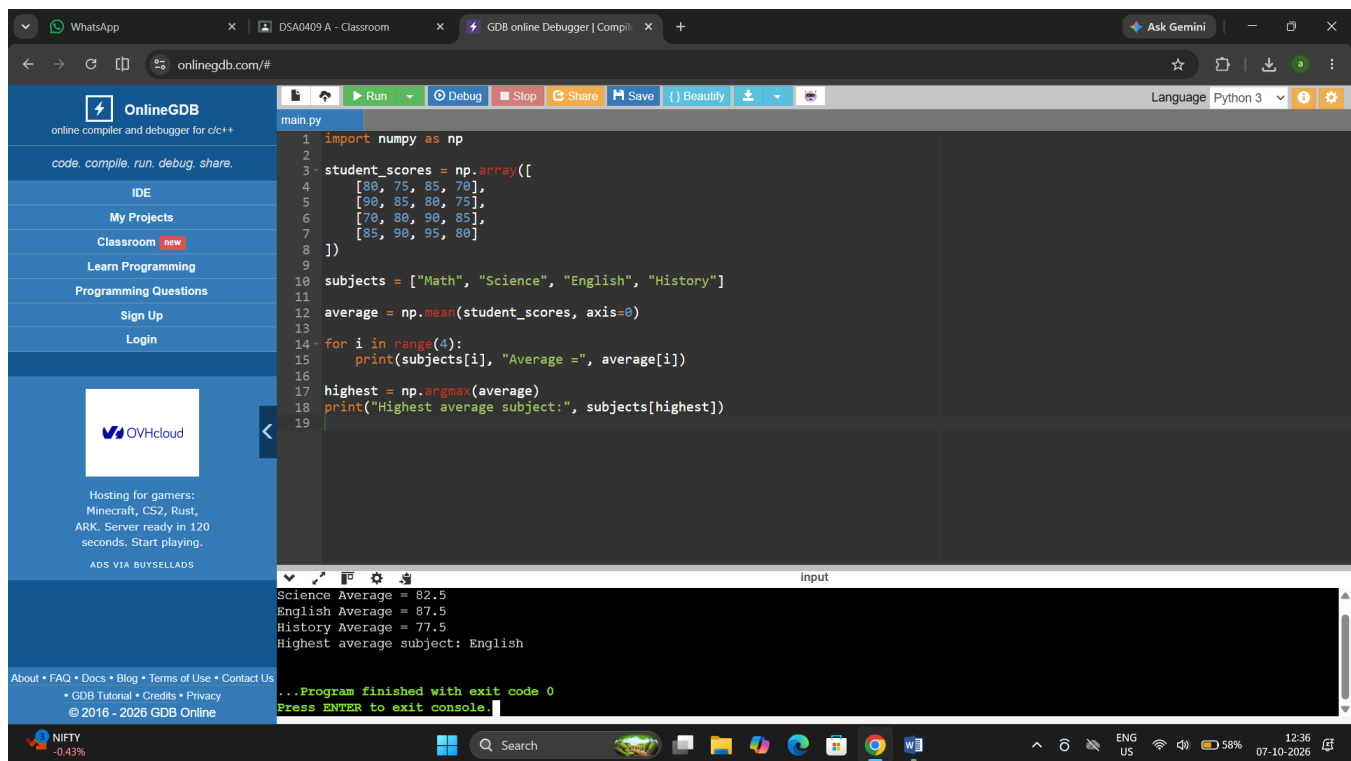
subjects = ["Math", "Science", "English", "History"]

average = np.mean(student_scores, axis=0)

for i in range(4):
    print(subjects[i], "Average =", average[i])

highest = np.argmax(average)
print("Highest average subject:", subjects[highest])
```

Output:



The screenshot shows the OnlineGDB web interface. The code editor contains the same Python code as shown above. The output console at the bottom displays the following results:

```
Science Average = 82.5
English Average = 87.5
History Average = 77.5
Highest average subject: English

...Program finished with exit code 0
Press ENTER to exit console
```

The interface also includes a sidebar with navigation links (OnlineGDB, IDE, My Projects, Classroom, Learn Programming, Programming Questions, Sign Up, Login) and a footer with site information.

Question 2: Average Product Price using NumPy

Program code:

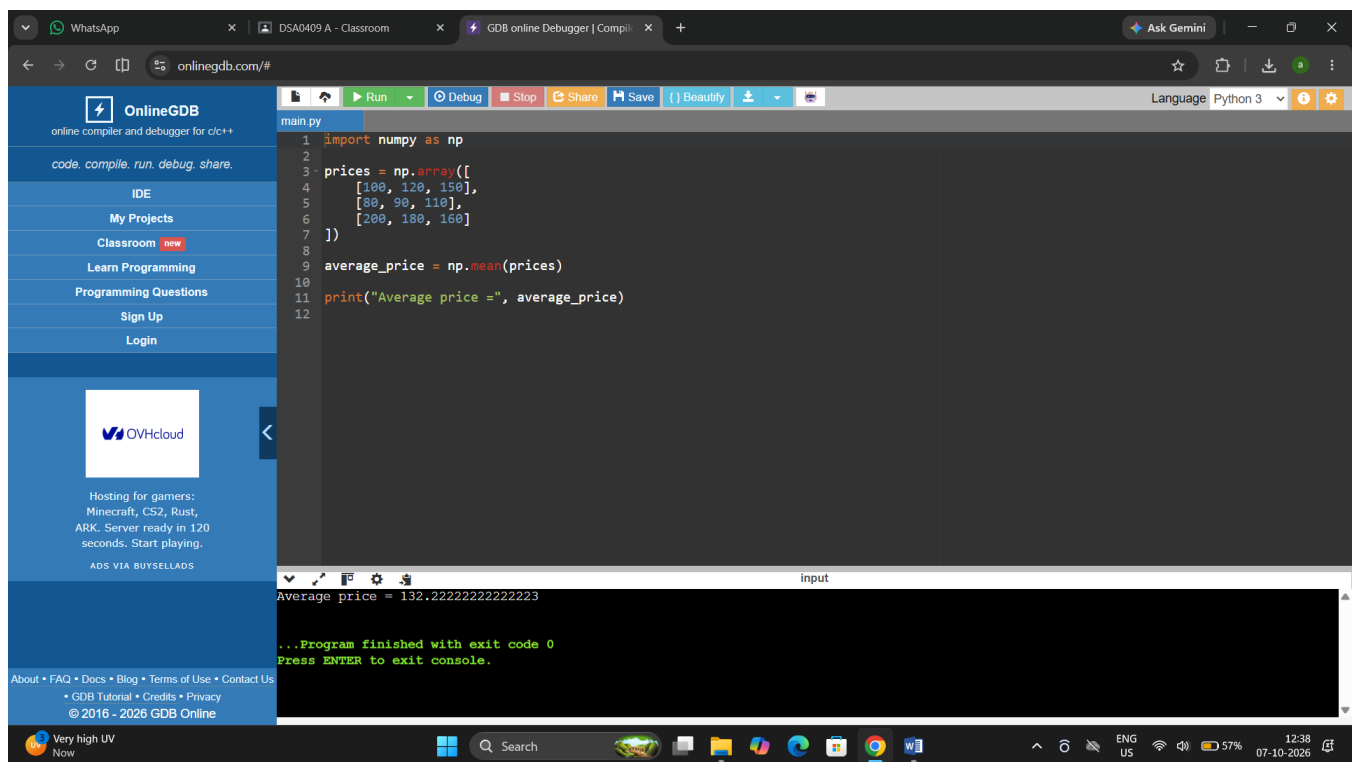
```
import numpy as np

prices = np.array([
    [100, 120, 150],
    [80, 90, 110],
    [200, 180, 160]
])

average_price = np.mean(prices)

print("Average price =", average_price)
```

Output:



The screenshot shows the OnlineGDB web interface. The left sidebar contains navigation links: OnlineGDB, code.compile.run.debug.share., IDE, My Projects, Classroom, Learn Programming, Programming Questions, Sign Up, and Login. The main editor area displays the Python code from the previous block. The bottom console shows the output: 'Average price = 132.22222222222223' followed by '...Program finished with exit code 0' and 'Press ENTER to exit console.' The browser's taskbar at the bottom shows the system clock as 12:38 on 07-10-2026.

```
main.py
1 import numpy as np
2
3 prices = np.array([
4     [100, 120, 150],
5     [80, 90, 110],
6     [200, 180, 160]
7 ])
8
9 average_price = np.mean(prices)
10
11 print("Average price =", average_price)
12
```

input

Average price = 132.22222222222223

...Program finished with exit code 0
Press ENTER to exit console.

Question 3: Average Sale Price of Houses with More Than 4 Bedrooms

Program code:

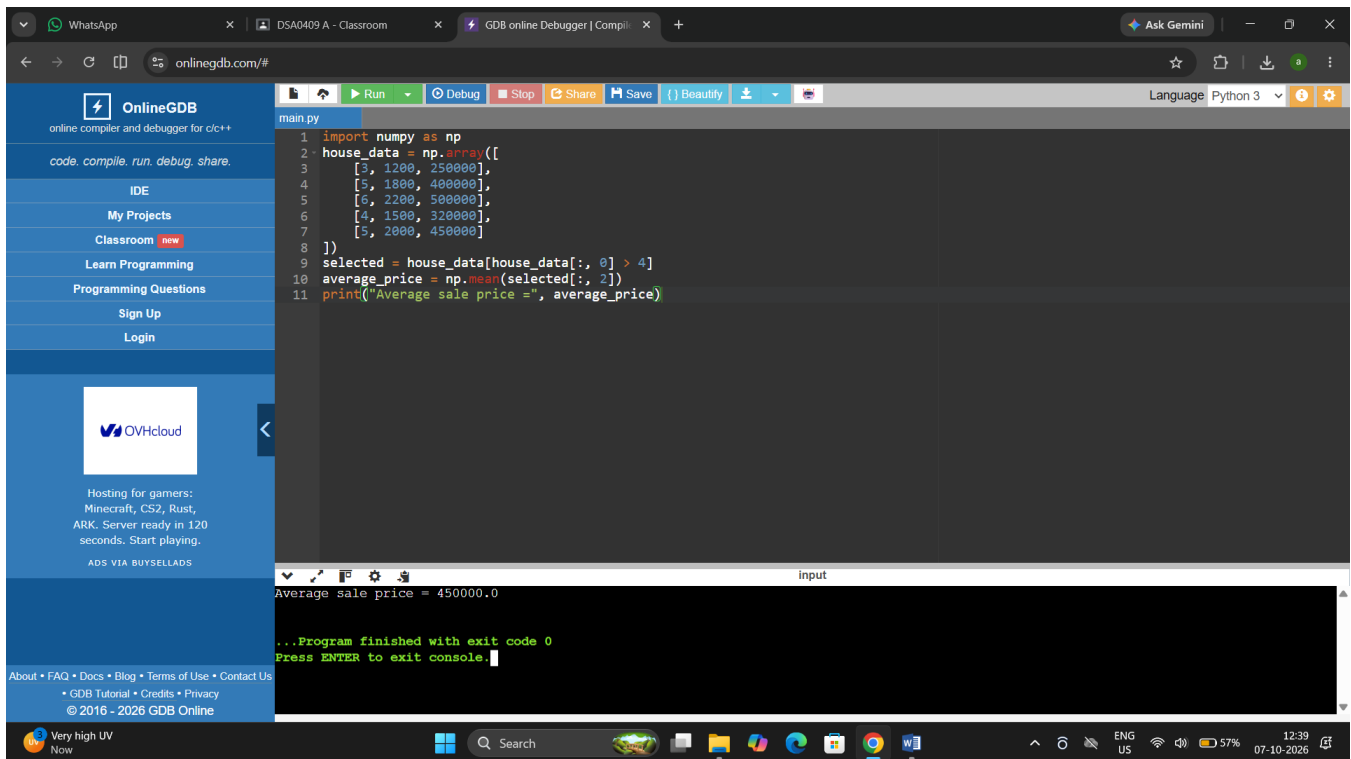
```
import numpy as np

# Columns: Bedrooms, Area, Sale Price
house_data = np.array([
    [3, 1200, 250000],
    [5, 1800, 400000],
    [6, 2200, 500000],
    [4, 1500, 320000],
    [5, 2000, 450000]
])

selected = house_data[house_data[:, 0] > 4]
average_price = np.mean(selected[:, 2])

print("Average sale price =", average_price)
```

Output:



The screenshot shows the OnlineGDB web interface. The code editor contains the following Python code:

```
1 import numpy as np
2 house_data = np.array([
3     [3, 1200, 250000],
4     [5, 1800, 400000],
5     [6, 2200, 500000],
6     [4, 1500, 320000],
7     [5, 2000, 450000]
8 ])
9 selected = house_data[house_data[:, 0] > 4]
10 average_price = np.mean(selected[:, 2])
11 print("Average sale price =", average_price)
```

The output console shows the result of the program execution:

```
input
Average sale price = 450000.0
...Program finished with exit code 0
Press ENTER to exit console
```

Question 4: Total Yearly Sales and Percentage Increase

Program code:

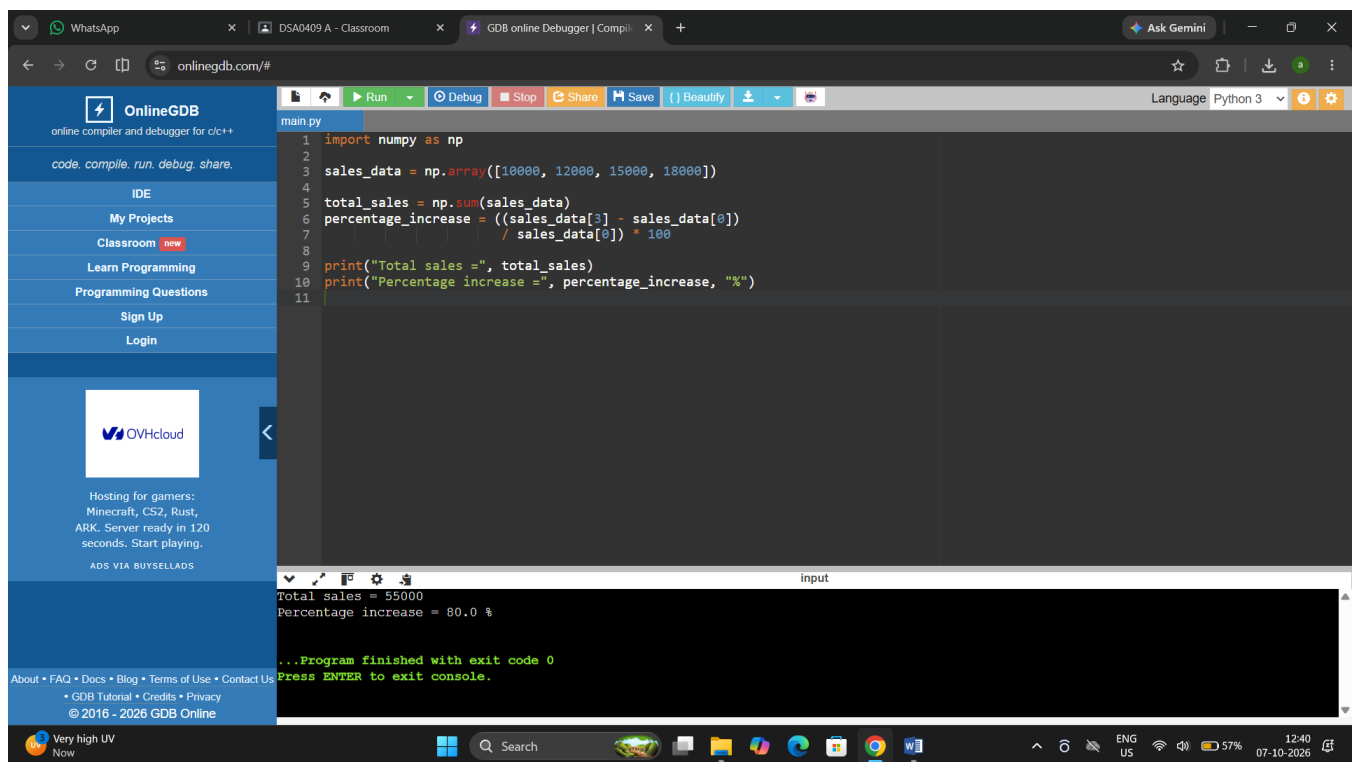
```
import numpy as np

sales_data = np.array([10000, 12000, 15000, 18000])

total_sales = np.sum(sales_data)
percentage_increase = ((sales_data[3] - sales_data[0])
                      / sales_data[0]) * 100

print("Total sales =", total_sales)
print("Percentage increase =", percentage_increase, "%")
```

Output:



The screenshot shows the OnlineGDB web interface. The left sidebar contains navigation links: OnlineGDB, code.compile.run.debug.share, IDE, My Projects, Classroom, Learn Programming, Programming Questions, Sign Up, and Login. The main editor area displays the Python code from the previous block. The bottom console shows the output: 'Total sales = 55000' and 'Percentage increase = 80.0 %'. The status bar at the bottom indicates 'Program finished with exit code 0' and 'Press ENTER to exit console.'.

```
main.py
1 import numpy as np
2
3 sales_data = np.array([10000, 12000, 15000, 18000])
4
5 total_sales = np.sum(sales_data)
6 percentage_increase = ((sales_data[3] - sales_data[0])
7                       / sales_data[0]) * 100
8
9 print("Total sales =", total_sales)
10 print("Percentage increase =", percentage_increase, "%")
11
```

input

```
Total sales = 55000
Percentage increase = 80.0 %

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 5: Average Fuel Efficiency and Percentage Improvement

Program code:

```
import numpy as np

fuel_efficiency = np.array([25, 30, 28, 35, 32])

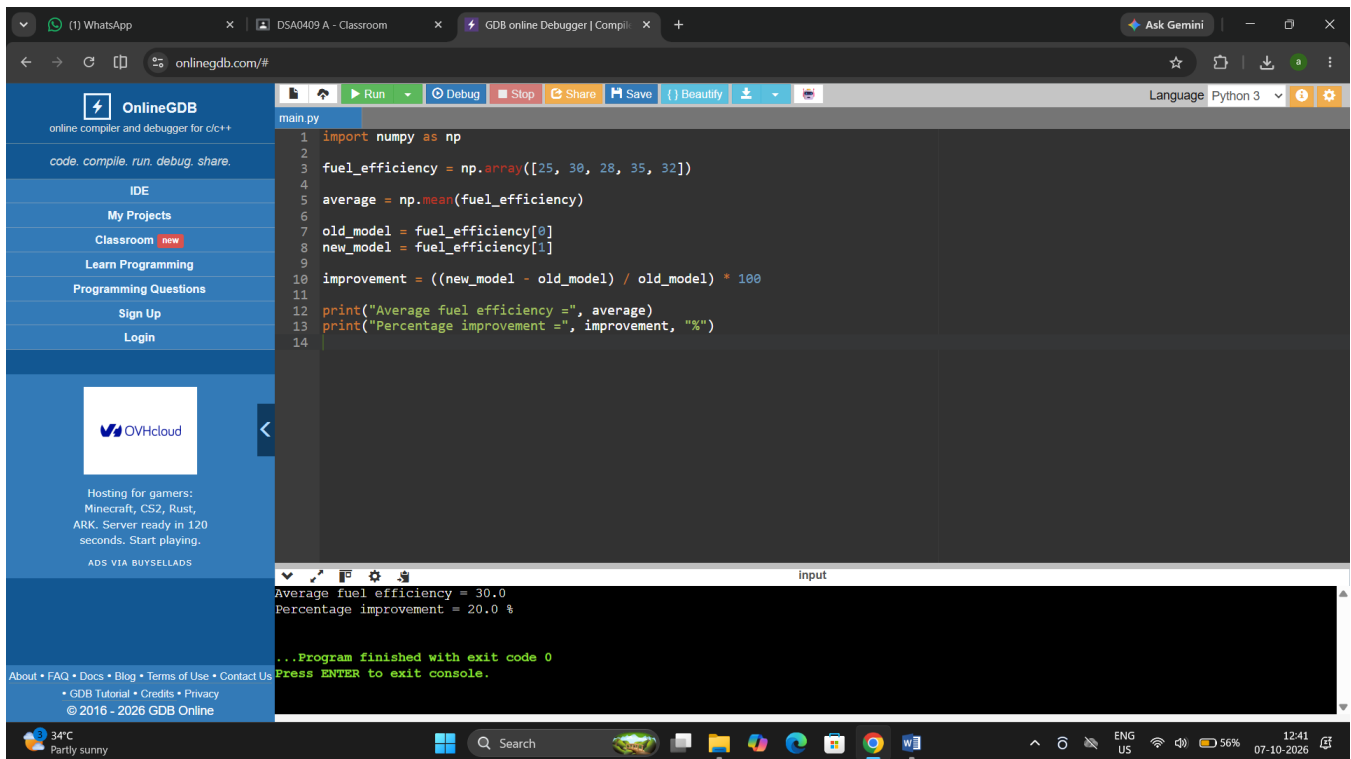
average = np.mean(fuel_efficiency)

old_model = fuel_efficiency[0]
new_model = fuel_efficiency[1]

improvement = ((new_model - old_model) / old_model) * 100

print("Average fuel efficiency =", average)
print("Percentage improvement =", improvement, "%")
```

Output:



The screenshot displays the OnlineGDB web interface. The left sidebar contains navigation links: OnlineGDB, code, compile, run, debug, share, IDE, My Projects, Classroom, Learn Programming, Programming Questions, Sign Up, and Login. The main editor area shows a Python script with the following code:

```
1 import numpy as np
2 fuel_efficiency = np.array([25, 30, 28, 35, 32])
3
4 average = np.mean(fuel_efficiency)
5
6
7 old_model = fuel_efficiency[0]
8 new_model = fuel_efficiency[1]
9
10 improvement = ((new_model - old_model) / old_model) * 100
11
12 print("Average fuel efficiency =", average)
13 print("Percentage improvement =", improvement, "%")
14
```

The output console at the bottom shows the following results:

```
Average fuel efficiency = 30.0
Percentage improvement = 20.0 %

...Program finished with exit code 0
Press ENTER to exit console.
```

The bottom status bar indicates the system temperature is 34°C, the weather is partly sunny, and the time is 12:41 on 07-10-2026.

Question 6: Total Grocery Cost with Discount and Tax

Program code:

```
prices = [100, 50, 200]
quantities = [2, 3, 1]

discount_rate = 10
tax_rate = 5

subtotal = 0

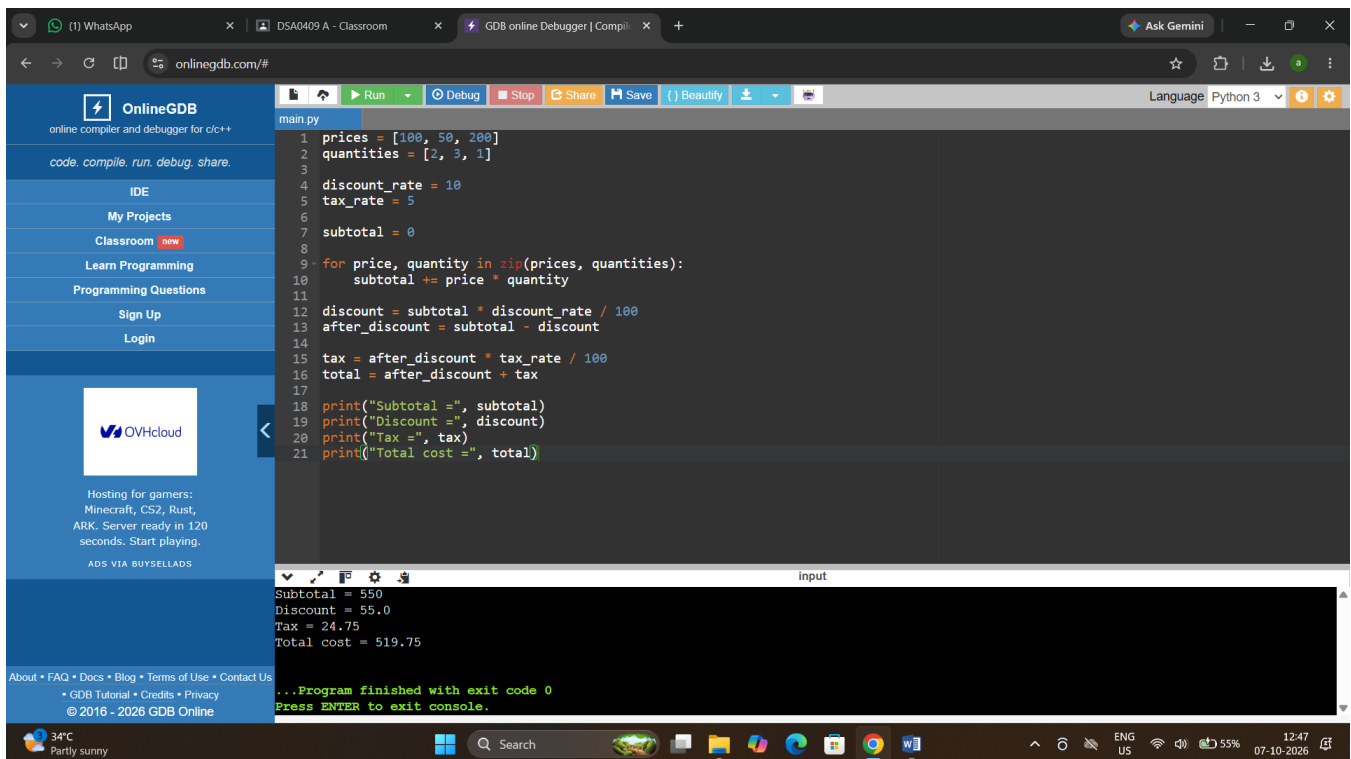
for price, quantity in zip(prices, quantities):
    subtotal += price * quantity

discount = subtotal * discount_rate / 100
after_discount = subtotal - discount

tax = after_discount * tax_rate / 100
total = after_discount + tax

print("Subtotal =", subtotal)
print("Discount =", discount)
print("Tax =", tax)
print("Total cost =", total)
```

Output:



The screenshot shows the OnlineGDB web interface. The left sidebar contains navigation links: OnlineGDB, code, compile, run, debug, share, IDE, My Projects, Classroom, Learn Programming, Programming Questions, Sign Up, and Login. The main editor area displays the Python code from the previous block. The bottom console shows the output of the program:

```
Subtotal = 550
Discount = 55.0
Tax = 24.75
Total cost = 519.75

...Program finished with exit code 0
Press ENTER to exit console.
```

The browser's taskbar at the bottom shows the system date and time as 12:47 on 07-10-2026, along with various system icons.

Question 7: Pandas Order Analysis

Program code:

```
import pandas as pd

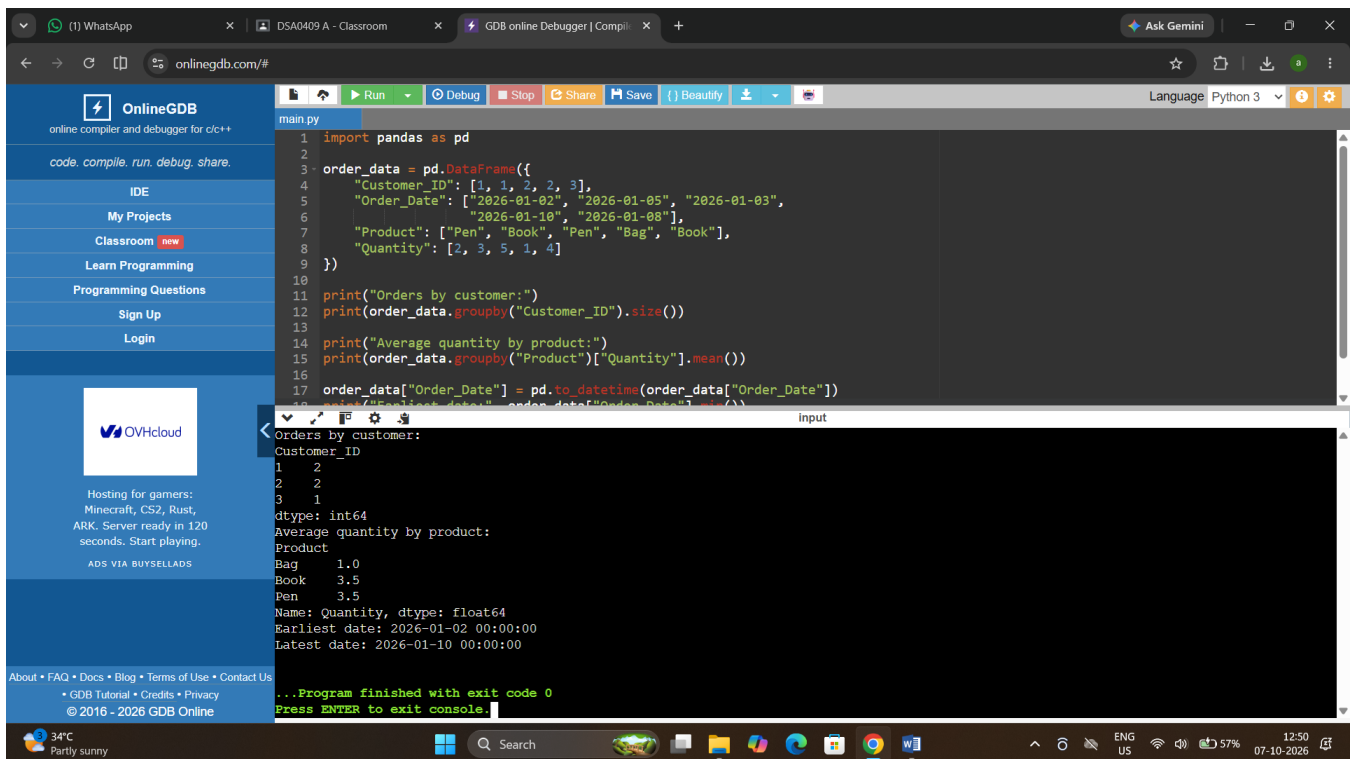
order_data = pd.DataFrame({
    "Customer_ID": [1, 1, 2, 2, 3],
    "Order_Date": ["2026-01-02", "2026-01-05", "2026-01-03",
                  "2026-01-10", "2026-01-08"],
    "Product": ["Pen", "Book", "Pen", "Bag", "Book"],
    "Quantity": [2, 3, 5, 1, 4]
})

print("Orders by customer:")
print(order_data.groupby("Customer_ID").size())

print("Average quantity by product:")
print(order_data.groupby("Product")["Quantity"].mean())

order_data["Order_Date"] = pd.to_datetime(order_data["Order_Date"])
print("Earliest date:", order_data["Order_Date"].min())
print("Latest date:", order_data["Order_Date"].max())
```

Output:



The screenshot shows a web browser window with the OnlineGDB interface. The code editor contains the Python script from the previous block. The console output is as follows:

```
Orders by customer:
Customer_ID
1      2
2      2
3      1
dtype: int64
Average quantity by product:
Product
Bag      1.0
Book     3.5
Pen      3.5
Name: Quantity, dtype: float64
Earliest date: 2026-01-02 00:00:00
Latest date: 2026-01-10 00:00:00

...Program finished with exit code 0
Press ENTER to exit console
```

Question 8: Top 5 Products Sold

Program code:

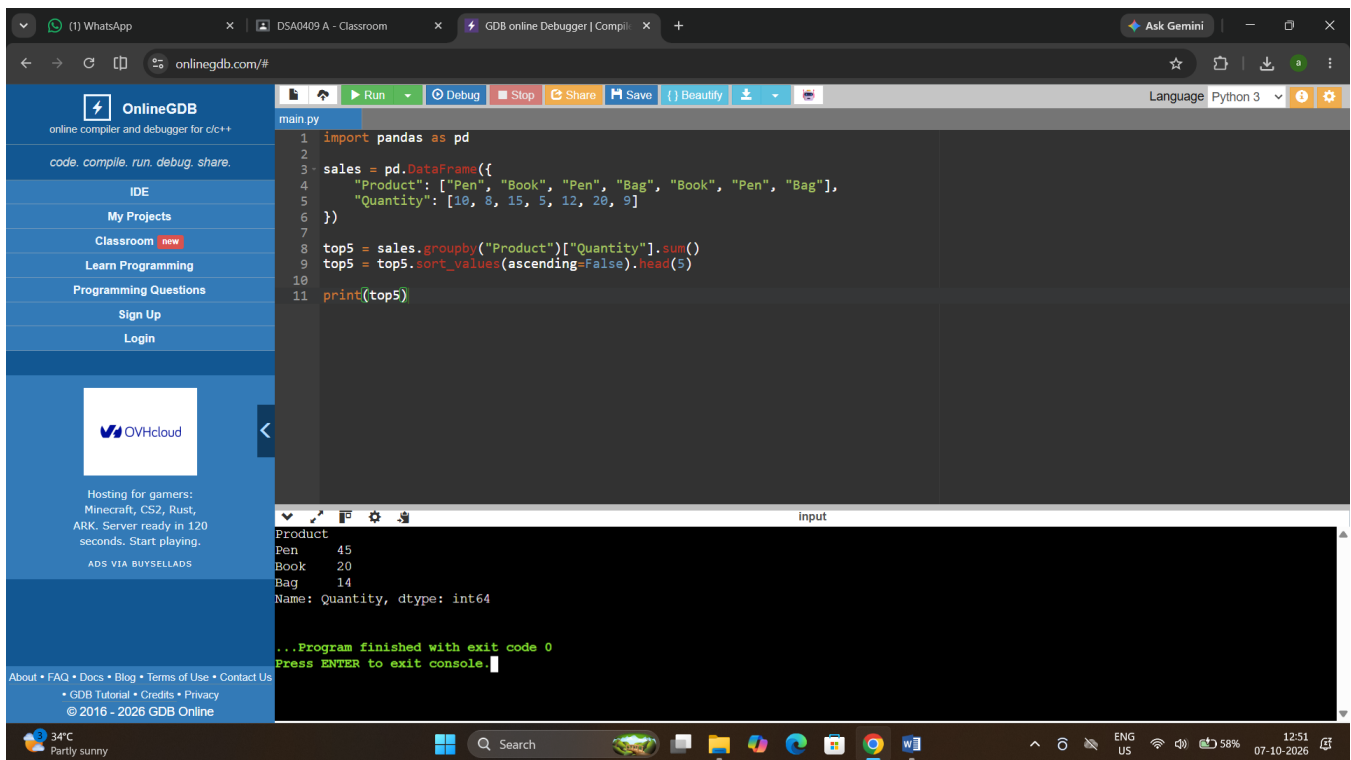
```
import pandas as pd

sales = pd.DataFrame({
    "Product": ["Pen", "Book", "Pen", "Bag", "Book", "Pen", "Bag"],
    "Quantity": [10, 8, 15, 5, 12, 20, 9]
})

top5 = sales.groupby("Product")["Quantity"].sum()
top5 = top5.sort_values(ascending=False).head(5)

print(top5)
```

Output:



The screenshot shows the OnlineGDB web interface. The code editor contains the following Python code:

```
1 import pandas as pd
2
3 sales = pd.DataFrame({
4     "Product": ["Pen", "Book", "Pen", "Bag", "Book", "Pen", "Bag"],
5     "Quantity": [10, 8, 15, 5, 12, 20, 9]
6 })
7
8 top5 = sales.groupby("Product")["Quantity"].sum()
9 top5 = top5.sort_values(ascending=False).head(5)
10
11 print(top5)
```

The output console shows the following results:

```
Product
Pen      45
Book     20
Bag      14
Name: Quantity, dtype: int64

...Program finished with exit code 0
Press ENTER to exit console
```

The interface also includes a sidebar with navigation links (IDE, My Projects, Classroom, Learn Programming, Programming Questions, Sign Up, Login) and a footer with site information (About, FAQ, Docs, Blog, Terms of Use, Contact Us, GDB Tutorial, Credits, Privacy, © 2016 - 2026 GDB Online).

Question 9: Property Data Analysis using Pandas

Program code:

```
import pandas as pd

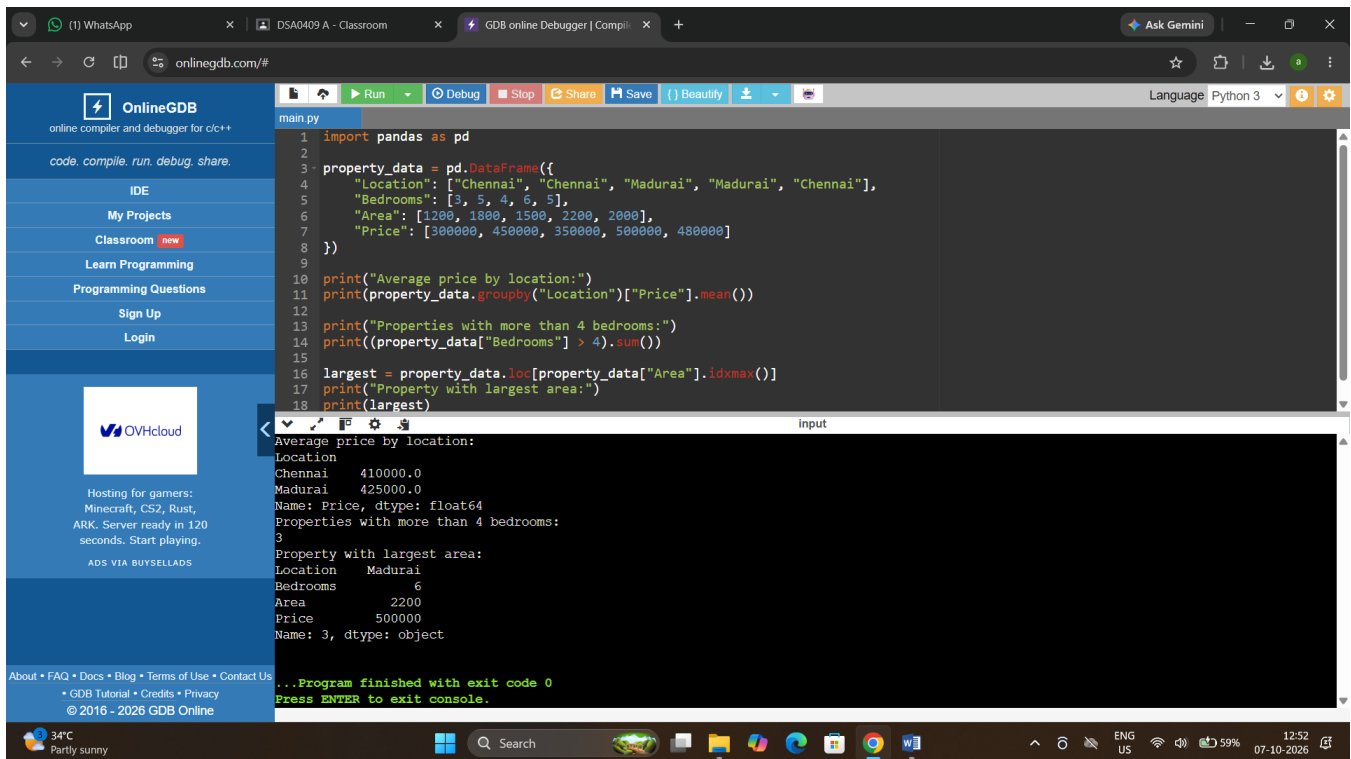
property_data = pd.DataFrame({
    "Location": ["Chennai", "Chennai", "Madurai", "Madurai", "Chennai"],
    "Bedrooms": [3, 5, 4, 6, 5],
    "Area": [1200, 1800, 1500, 2200, 2000],
    "Price": [300000, 450000, 350000, 500000, 480000]
})

print("Average price by location:")
print(property_data.groupby("Location")["Price"].mean())

print("Properties with more than 4 bedrooms:")
print((property_data["Bedrooms"] > 4).sum())

largest = property_data.loc[property_data["Area"].idxmax()]
print("Property with largest area:")
print(largest)
```

Output:



```
main.py
1 import pandas as pd
2
3 property_data = pd.DataFrame({
4     "Location": ["Chennai", "Chennai", "Madurai", "Madurai", "Chennai"],
5     "Bedrooms": [3, 5, 4, 6, 5],
6     "Area": [1200, 1800, 1500, 2200, 2000],
7     "Price": [300000, 450000, 350000, 500000, 480000]
8 })
9
10 print("Average price by location:")
11 print(property_data.groupby("Location")["Price"].mean())
12
13 print("Properties with more than 4 bedrooms:")
14 print((property_data["Bedrooms"] > 4).sum())
15
16 largest = property_data.loc[property_data["Area"].idxmax()]
17 print("Property with largest area:")
18 print(largest)

Average price by location:
Location
Chennai    410000.0
Madurai    425000.0
Name: Price, dtype: float64
Properties with more than 4 bedrooms:
3
Property with largest area:
Location    Madurai
Bedrooms      6
Area        2200
Price      500000
Name: 3, dtype: object

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 10: Monthly Sales – Line Plot and Bar Plot

Program code:

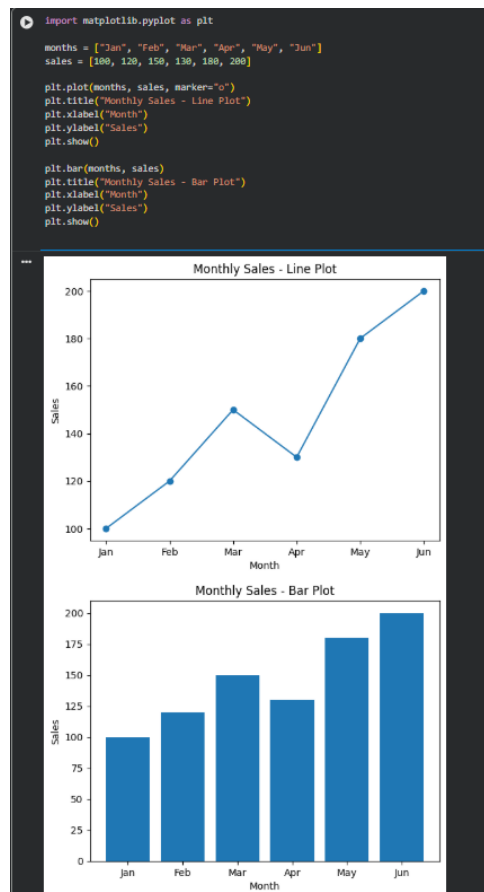
```
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
sales = [100, 120, 150, 130, 180, 200]

plt.plot(months, sales, marker="o")
plt.title("Monthly Sales - Line Plot")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()

plt.bar(months, sales)
plt.title("Monthly Sales - Bar Plot")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()
```

Output:



Question 11: Sales over Time – Line, Scatter and Bar Plots

Program code:

```
import matplotlib.pyplot as plt

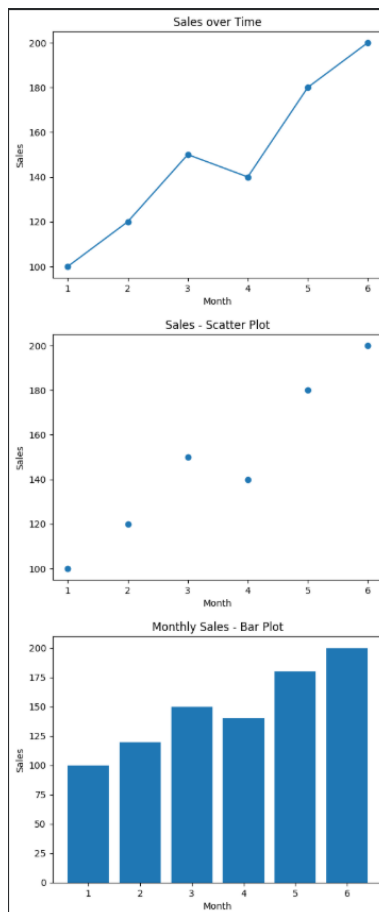
months = [1, 2, 3, 4, 5, 6]
sales = [100, 120, 150, 140, 180, 200]

plt.plot(months, sales, marker="o")
plt.title("Sales over Time")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()

plt.scatter(months, sales)
plt.title("Sales - Scatter Plot")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()

plt.bar(months, sales)
plt.title("Monthly Sales - Bar Plot")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()
```

Output:



Question 12: Monthly Temperature and Rainfall Plots

Program code:

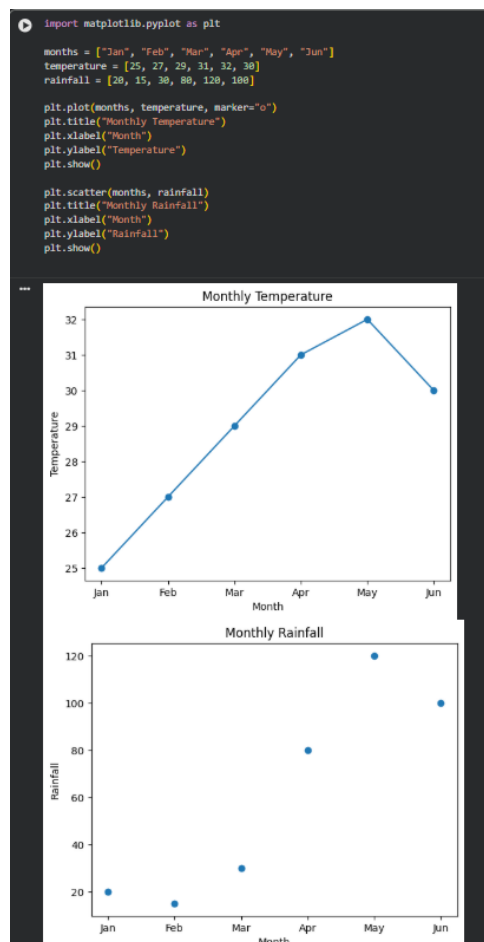
```
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
temperature = [25, 27, 29, 31, 32, 30]
rainfall = [20, 15, 30, 80, 120, 100]

plt.plot(months, temperature, marker="o")
plt.title("Monthly Temperature")
plt.xlabel("Month")
plt.ylabel("Temperature")
plt.show()

plt.scatter(months, rainfall)
plt.title("Monthly Rainfall")
plt.xlabel("Month")
plt.ylabel("Rainfall")
plt.show()
```

Output:



Question 13: Word Frequency from a Text File

Program code:

```
from collections import Counter
import re

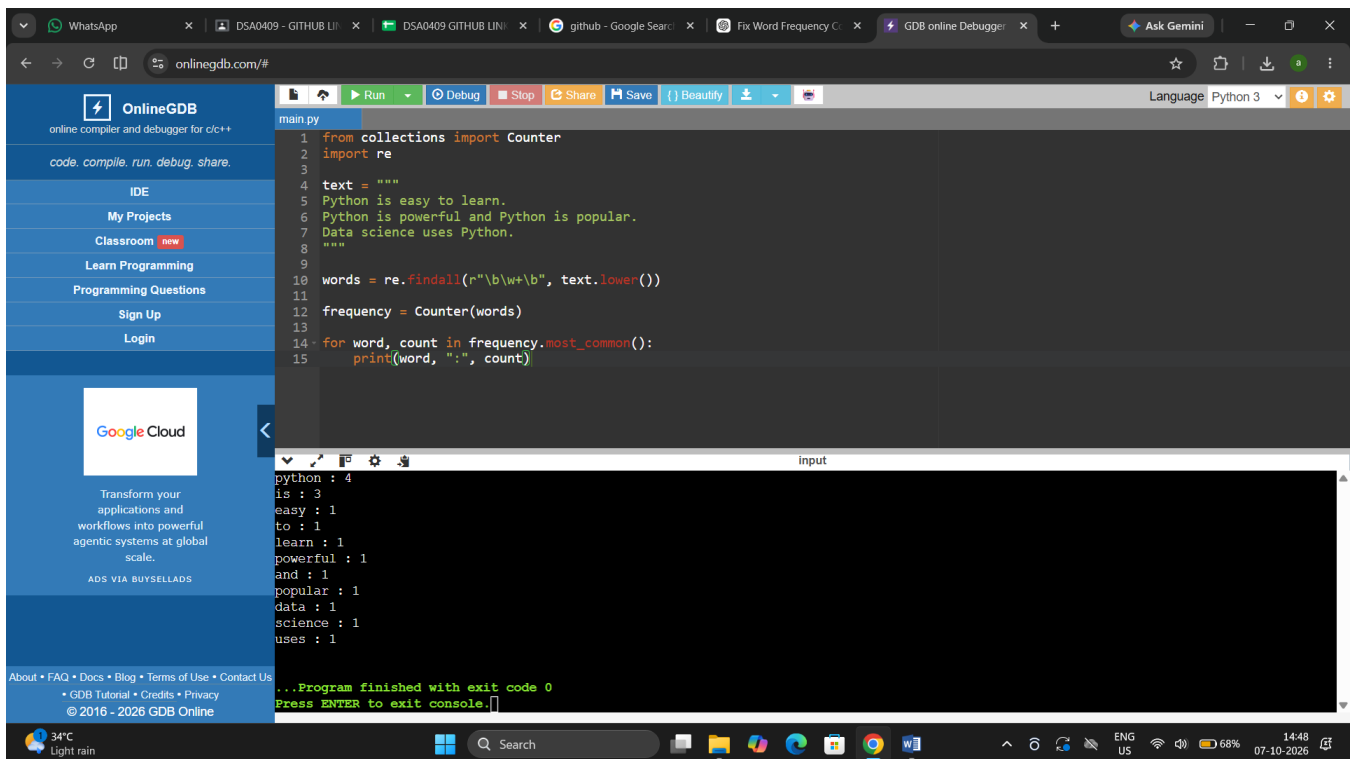
with open("sample_text.txt", "r") as file:
    text = file.read().lower()

words = re.findall(r"\b\w+\b", text)

frequency = Counter(words)

for word, count in frequency.most_common():
    print(word, ":", count)
```

Output:



The screenshot shows the OnlineGDB web interface. The left sidebar contains navigation links: OnlineGDB, code, compile, run, debug, share, IDE, My Projects, Classroom, Learn Programming, Programming Questions, Sign Up, and Login. The main editor area displays the Python code for word frequency analysis. The output console shows the results of the program execution.

```
python : 4
is : 3
easy : 1
to : 1
learn : 1
powerful : 1
and : 1
popular : 1
data : 1
science : 1
uses : 1

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 14: Frequency Distribution of Customer Ages

Program code:

```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.DataFrame({
    "Age": [20, 25, 20, 30, 25, 20, 35, 30, 25]
})

frequency = data["Age"].value_counts().sort_index()

print(frequency)

frequency.plot(kind="bar")
plt.title("Customer Age Frequency")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```

Output:



Question 15: Frequency Distribution of Likes

Program code:

```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.DataFrame({
    "Likes": [10, 20, 10, 30, 20, 10, 40, 30]
})

frequency = data["Likes"].value_counts().sort_index()

print(frequency)

frequency.plot(kind="bar")
plt.title("Frequency of Likes")
plt.xlabel("Likes")
plt.ylabel("Number of Posts")
plt.show()
```

Output:



Question 16: Word Frequency in Customer Reviews

Program code:

```
import pandas as pd
import re
from collections import Counter

reviews = pd.DataFrame({
    "Review": [
        "good product and good service",
        "excellent product",
        "good quality product"
    ]
})

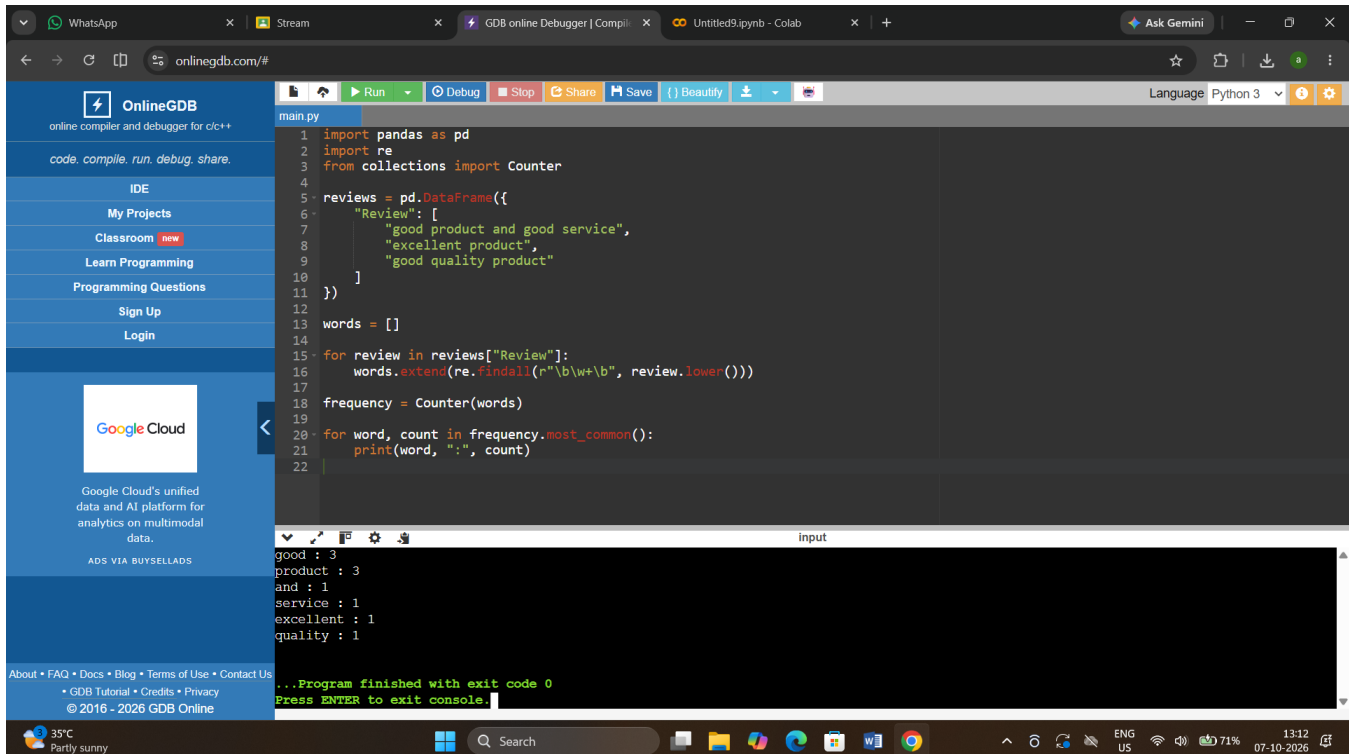
words = []

for review in reviews["Review"]:
    words.extend(re.findall(r"\b\w+\b", review.lower()))

frequency = Counter(words)

for word, count in frequency.most_common():
    print(word, ":", count)
```

Output:



The screenshot displays a web browser window with the OnlineGDB interface. The left sidebar contains navigation links like 'OnlineGDB', 'My Projects', and 'Classroom'. The main editor area shows the Python code from the previous block. Below the editor, the 'input' tab is active, showing the program's output: 'good : 3', 'product : 3', 'and : 1', 'service : 1', 'excellent : 1', and 'quality : 1'. At the bottom, a status bar indicates the program finished with exit code 0.

```
main.py
1 import pandas as pd
2 import re
3 from collections import Counter
4
5 reviews = pd.DataFrame({
6     "Review": [
7         "good product and good service",
8         "excellent product",
9         "good quality product"
10    ]
11 })
12
13 words = []
14
15 for review in reviews["Review"]:
16     words.extend(re.findall(r"\b\w+\b", review.lower()))
17
18 frequency = Counter(words)
19
20 for word, count in frequency.most_common():
21     print(word, ":", count)
22
```

input

```
good : 3
product : 3
and : 1
service : 1
excellent : 1
quality : 1
```

...Program finished with exit code 0
Press ENTER to exit console.

Question 17: Customer Feedback – Top N Word Frequency

Program code:

```
import pandas as pd
import re
from collections import Counter
import matplotlib.pyplot as plt

data = pd.read_csv("data.csv")

stop_words = {"the", "and", "is", "a", "an", "of", "to", "in"}

words = []

for text in data["feedback"].dropna():
    text = text.lower()
    words.extend(re.findall(r"\b[a-z]+\b", text))

words = [word for word in words if word not in stop_words]

frequency = Counter(words)

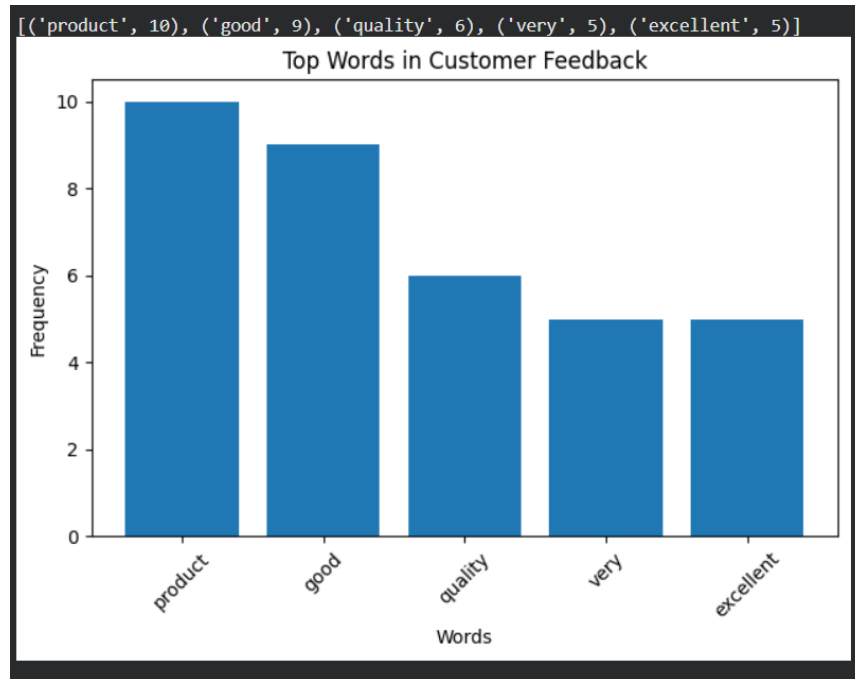
n = int(input("Enter N: "))
top_words = frequency.most_common(n)

print(top_words)

names = [x[0] for x in top_words]
counts = [x[1] for x in top_words]

plt.bar(names, counts)
plt.xticks(rotation=45)
plt.title("Top Words in Customer Feedback")
plt.show()
```

Output:



Question 18: Mean, Median, Standard Deviation, Boxplot, Scatter Plot and Q-Q Plot

Program code:

```
import pandas as pd
import matplotlib.pyplot as plt
import scipy.stats as stats

data = pd.DataFrame({
    "Age": [20, 22, 25, 28, 30, 32, 35, 40],
    "Fat": [15, 18, 20, 22, 24, 27, 30, 32]
})

print("Mean:")
print(data.mean())

print("Median:")
print(data.median())

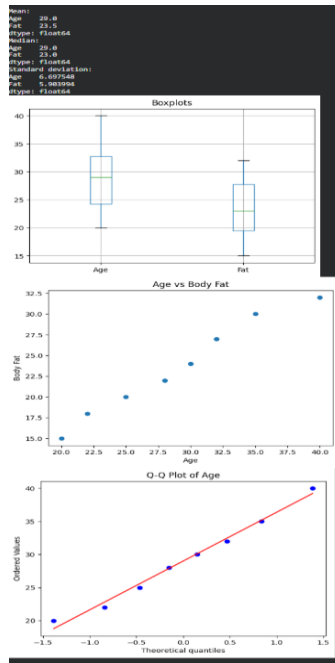
print("Standard deviation:")
print(data.std())

data[["Age", "Fat"]].boxplot()
plt.title("Boxplots")
plt.show()

plt.scatter(data["Age"], data["Fat"])
plt.xlabel("Age")
plt.ylabel("Body Fat")
plt.title("Age vs Body Fat")
plt.show()

stats.probplot(data["Age"], dist="norm", plot=plt)
plt.title("Q-Q Plot of Age")
plt.show()
```

Output:



Question 19: 95% Confidence Interval for Blood Pressure Reduction

Program code:

```

import numpy as np
from scipy import stats

drug = np.array([8, 10, 12, 9, 11, 13, 10, 12])
placebo = np.array([3, 5, 4, 6, 2, 5, 4, 3])

def confidence_interval(data):
    mean = np.mean(data)
    sem = stats.sem(data)
    interval = stats.t.interval(
        0.95, len(data) - 1, loc=mean, scale=sem
    )
    return interval

print("Drug 95% CI:", confidence_interval(drug))
print("Placebo 95% CI:", confidence_interval(placebo))

```

Output:

The screenshot shows the OnlineGDB web interface. The left sidebar contains navigation links: OnlineGDB, code, compile, run, debug, share, IDE, My Projects, Classroom, Learn Programming, Programming Questions, Sign Up, and Login. The main editor area displays a Python script named 'main.py' with the following code:

```
1 import numpy as np
2 from scipy import stats
3
4 drug = np.array([8, 10, 12, 9, 11, 13, 10, 12])
5 placebo = np.array([3, 5, 4, 6, 2, 5, 4, 3])
6
7 def confidence_interval(data):
8     mean = np.mean(data)
9     sem = stats.sem(data)
10    interval = stats.t.interval(
11        0.95, len(data) - 1, loc=mean, scale=sem
12    )
13    return interval
14
15 print("Drug 95% CI:", confidence_interval(drug))
16 print("Placebo 95% CI:", confidence_interval(placebo))
17
```

The output window shows the following results:

```
Drug 95% CI: (np.float64(9.216289685290807), np.float64(12.033710314709193))
Placebo 95% CI: (np.float64(2.9053916697328868), np.float64(5.094608330267113))
...Program finished with exit code 0
Press ENTER to exit console.
```

Question 20: A/B Test – Compare Two Conversion Rate Samples

Program code:

```
import numpy as np
from scipy.stats import ttest_ind

design_A = np.array([0.10, 0.12, 0.11, 0.13, 0.09, 0.12])
design_B = np.array([0.14, 0.15, 0.13, 0.16, 0.12, 0.14])

t_stat, p_value = ttest_ind(design_A, design_B)

print("t-statistic =", t_stat)
print("p-value =", p_value)

if p_value < 0.05:
    print("Statistically significant difference")
else:
    print("No statistically significant difference")
```

Output:

The screenshot shows the OnlineGDB web interface. The left sidebar contains navigation links: OnlineGDB, code, compile, run, debug, share, IDE, My Projects, Classroom, Learn Programming, Programming Questions, Sign Up, and Login. The main editor area displays a Python script named 'main.py' using the 'Python 3' language. The script imports 'numpy' and 'scipy.stats', defines two arrays 'design_A' and 'design_B', and performs a t-test using 'ttest_ind'. The output in the console shows the t-statistic, p-value, and a message indicating a statistically significant difference. The bottom status bar shows the system time as 13:15 on 07-10-2026.

```
1 import numpy as np
2 from scipy.stats import ttest_ind
3
4 design_A = np.array([0.10, 0.12, 0.11, 0.13, 0.09, 0.12])
5 design_B = np.array([0.14, 0.15, 0.13, 0.16, 0.12, 0.14])
6
7 t_stat, p_value = ttest_ind(design_A, design_B)
8
9 print("t-statistic =", t_stat)
10 print("p-value =", p_value)
11
12 if p_value < 0.05:
13     print("Statistically significant difference")
14 else:
15     print("No statistically significant difference")
16
```

input

```
t-statistic = -3.3999999999999995
p-value = 0.006771072424337956
Statistically significant difference

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 21: Point Estimate and Confidence Interval from CSV

Program code:

```
import pandas as pd
import numpy as np
from scipy import stats

data = pd.read_csv("rare_elements.csv")
values = data.iloc[:, 0].dropna()

n = int(input("Enter sample size: "))
confidence = float(input("Enter confidence level (for example 0.95): "))
precision = float(input("Enter desired precision: "))

sample = values.sample(n=min(n, len(values)), random_state=1)

mean = sample.mean()
sem = stats.sem(sample)

ci = stats.t.interval(
    confidence, len(sample) - 1, loc=mean, scale=sem
)
```

```

print("Point estimate =", mean)
print("Confidence interval =", ci)
print("Desired precision =", precision)

```

Output:

The screenshot shows the OnlineGDB web interface. The left sidebar contains navigation links: IDE, My Projects, Classroom (highlighted), Learn Programming, Programming Questions, Sign Up, and Login. The main editor area displays a Python script in a dark-themed editor. The script imports pandas and scipy, reads a CSV file, and calculates a confidence interval. The output is displayed in the console at the bottom.

```

main.py
9 16.4, 13.6, 14.9, 15.1, 12.5
10 ]
11 })
12
13 values = data.iloc[:, 0].dropna()
14
15 n = 10
16 confidence = 0.95
17 precision = 0.5
18
19 sample = values.sample(n=min(n, len(values)), random_state=1)
20
21 mean = sample.mean()
22 sem = stats.sem(sample)
23
24 ci = stats.t.interval(
25     confidence,
26     len(sample) - 1,
27     loc=mean,
28     scale=sem
29 )
30
31 print("Point estimate =", mean)
32 print("Confidence interval =", ci)
33 print("Desired precision =", precision)

```

input

```

Point estimate = 14.49
Confidence interval = (np.float64(13.44927111035885), np.float64(15.53072888964115))
Desired precision = 0.5

...Program finished with exit code 0
Press ENTER to exit console

```

Question 22: Confidence Interval for Mean Customer Rating

Program code:

```

import pandas as pd
from scipy import stats

data = pd.read_csv("customer_reviews.csv")
ratings = data["rating"].dropna()

confidence = 0.95
mean = ratings.mean()
sem = stats.sem(ratings)

ci = stats.t.interval(
    confidence, len(ratings) - 1,
    loc=mean, scale=sem
)

print("Mean rating =", mean)
print("95% Confidence interval =", ci)

```

Output:

The screenshot shows the OnlineGDB web interface. The left sidebar contains navigation links: OnlineGDB, code.compile.run.debug.share, IDE, My Projects, Classroom, Learn Programming, Programming Questions, Sign Up, and Login. The main editor area displays a Python script in a dark-themed editor. The script imports pandas and scipy, creates a DataFrame with 'rating' data, drops the 'rating' column, calculates the mean, standard error, and 95% confidence interval, and prints the results. The output console at the bottom shows the calculated mean rating and confidence interval.

```
main.py
1 import pandas as pd
2 from scipy import stats
3
4 data = pd.DataFrame({
5     "rating": [
6         4, 5, 3, 4, 5,
7         4, 3, 5, 4, 5,
8         4, 4, 5, 3, 4
9     ]
10 })
11
12 ratings = data["rating"].dropna()
13
14 confidence = 0.95
15 mean = ratings.mean()
16 sem = stats.sem(ratings)
17
18 ci = stats.t.interval(
19     confidence,
20     xer(ratings) - 1,
21     loc=mean,
22     scale=sem
23 )
24
25 print("Mean rating =", mean)
26 print("95% Confidence interval =", ci)
```

input

```
Mean rating = 4.133333333333334
95% Confidence interval = (np.float64(3.7217499592058503), np.float64(4.544916707460818))

...Program finished with exit code 0
Press ENTER to exit console
```

Question 23: Hypothesis Testing for Treatment vs Placebo

Program code:

```
import numpy as np
from scipy.stats import ttest_ind
import matplotlib.pyplot as plt

control = np.array([5, 6, 7, 5, 6, 4, 7])
treatment = np.array([8, 9, 10, 9, 11, 8, 10])

t_stat, p_value = ttest_ind(treatment, control)

print("t-statistic =", t_stat)
print("p-value =", p_value)

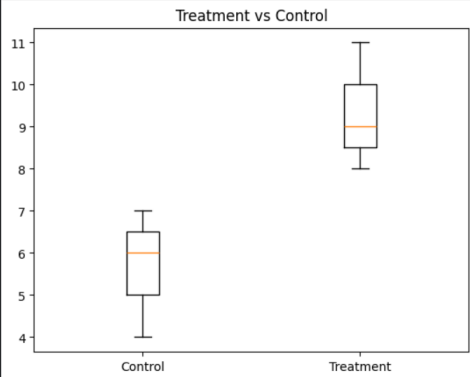
if p_value < 0.05:
    print("Significant difference")
else:
    print("No significant difference")

plt.boxplot([control, treatment], labels=["Control", "Treatment"])
```

```
plt.title("Treatment vs Control")
plt.show()
```

Output:

```
t-statistic = 6.004805767690768
p-value = 6.171074121437904e-05
Significant difference
/tmp/ipykernel_511/1303493643.py:18: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in the future.
  plt.boxplot([control, treatment], labels=["Control", "Treatment"])
```



Question 24: KNN Classifier for a New Patient

Program code:

```
from sklearn.neighbors import KNeighborsClassifier

# Features: symptom1, symptom2
X = [
    [1, 1], [1, 2], [2, 1],
    [8, 8], [9, 8], [8, 9]
]

# 0 = No condition, 1 = Condition
y = [0, 0, 0, 1, 1, 1]

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X, y)

patient = [[7, 8]]

prediction = model.predict(patient)
```

```

if prediction[0] == 1:
    print("Patient may have the condition")
else:
    print("Patient may not have the condition")

```

Output:

The screenshot shows the OnlineGDB web interface. The code editor contains the following Python code:

```

1 from sklearn.neighbors import KNeighborsClassifier
2
3 # Features: symptom1, symptom2
4 X = [
5     [1, 1], [1, 2], [2, 1],
6     [8, 8], [9, 8], [8, 9]
7 ]
8
9 # 0 = No condition, 1 = Condition
10 y = [0, 0, 0, 1, 1, 1]
11
12 model = KNeighborsClassifier(n_neighbors=3)
13 model.fit(X, y)
14
15 patient = [[7, 8]]
16
17 prediction = model.predict(patient)
18
19 if prediction[0] == 1:
20     print("Patient may have the condition")
21 else:
22     print("Patient may not have the condition")
23

```

The console output shows:

```

input
Patient may have the condition
...Program finished with exit code 0
Press ENTER to exit console

```

Question 25: Decision Tree for Iris Classification

Program code:

```

from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

iris = load_iris()

X = iris.data
y = iris.target

model = DecisionTreeClassifier(random_state=1)
model.fit(X, y)

sepal_length = float(input("Sepal length: "))
sepal_width = float(input("Sepal width: "))
petal_length = float(input("Petal length: "))
petal_width = float(input("Petal width: "))

new_flower = [
    sepal_length, sepal_width,
    petal_length, petal_width
]

```

```

]]

prediction = model.predict(new_flower)

print("Predicted species:", iris.target_names[prediction[0]])

```

Output:

The screenshot shows the OnlineGDB web interface. The left sidebar contains navigation links like 'IDE', 'My Projects', 'Classroom', 'Learn Programming', 'Programming Questions', 'Sign Up', and 'Login'. The main area displays a Python script for Iris dataset classification. The script imports sklearn datasets and DecisionTreeClassifier, loads the Iris dataset, fits a model, and takes user input for sepal, petal, and petal widths to predict a species. The output shows the predicted species as 'versicolor'.

```

main.py
1 from sklearn.datasets import load_iris
2 from sklearn.tree import DecisionTreeClassifier
3
4 iris = load_iris()
5
6 X = iris.data
7 y = iris.target
8
9 model = DecisionTreeClassifier(random_state=1)
10 model.fit(X, y)
11
12 sepal_length = float(input("Sepal length: "))
13 sepal_width = float(input("Sepal width: "))
14 petal_length = float(input("Petal length: "))
15 petal_width = float(input("Petal width: "))
16
17 new_flower = [[
18     sepal_length, sepal_width,
19     petal_length, petal_width
20 ]]
21
22 prediction = model.predict(new_flower)
23
24 print("Predicted species:", iris.target_names[prediction[0]])
25

```

input

```

Sepal length: 2
Sepal width: 3
Petal length: 1
Petal width: 3
Predicted species: versicolor

```

...Program finished with exit code 0
Press ENTER to exit console

Question 26: Linear Regression for Housing Price Prediction

Program code:

```

import numpy as np
from sklearn.linear_model import LinearRegression

# Area in square feet
X = np.array([[1000], [1200], [1500], [1800], [2000]])
# House prices
y = np.array([200000, 240000, 300000, 360000, 400000])

model = LinearRegression()
model.fit(X, y)

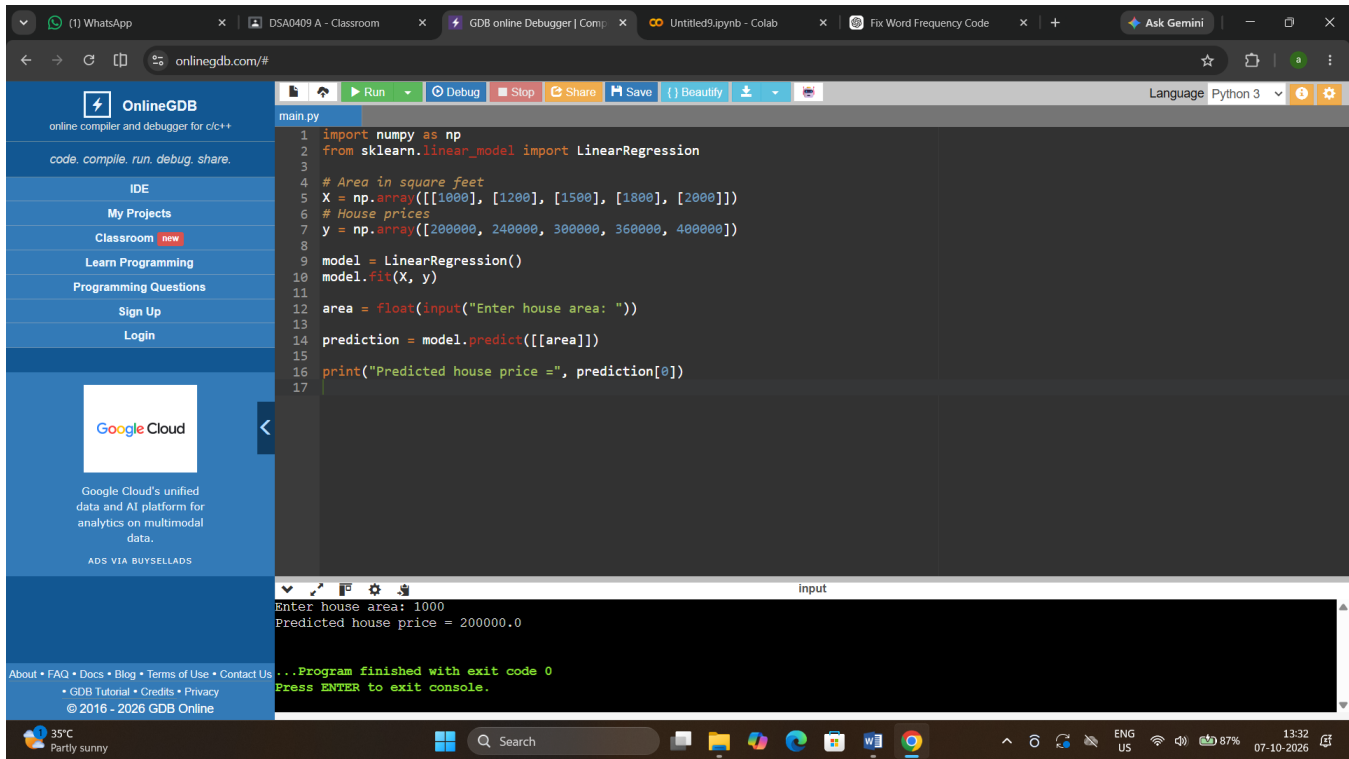
area = float(input("Enter house area: "))

prediction = model.predict([[area]])

print("Predicted house price =", prediction[0])

```

Output:



The screenshot shows the OnlineGDB web interface. The left sidebar contains navigation links: OnlineGDB, code.compile.run.debug.share, IDE, My Projects, Classroom, Learn Programming, Programming Questions, Sign Up, and Login. The main editor area displays a Python script for linear regression. The script imports numpy and sklearn, defines feature data (X) and target data (y), fits a LinearRegression model, and prompts the user to enter a house area to predict the price. The output console shows the user input '1000' and the predicted house price '200000.0'. The bottom status bar indicates the program finished with exit code 0.

```
main.py
1 import numpy as np
2 from sklearn.linear_model import LinearRegression
3
4 # Area in square feet
5 X = np.array([[1000], [1200], [1500], [1800], [2000]])
6 # House prices
7 y = np.array([200000, 240000, 300000, 360000, 400000])
8
9 model = LinearRegression()
10 model.fit(X, y)
11
12 area = float(input("Enter house area: "))
13
14 prediction = model.predict([[area]])
15
16 print("Predicted house price =", prediction[0])
17
```

input

```
Enter house area: 1000
Predicted house price = 200000.0

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 27: Logistic Regression for Customer Churn

Program code:

```
import numpy as np
from sklearn.linear_model import LogisticRegression

# Features: usage minutes, contract months
X = np.array([
    [100, 24], [120, 24], [80, 12],
    [300, 6], [280, 6], [350, 3]
])

# 0 = Not churned, 1 = Churned
y = np.array([0, 0, 0, 1, 1, 1])

model = LogisticRegression()
model.fit(X, y)

usage = float(input("Usage minutes: "))
```

```

contract = float(input("Contract duration: "))

prediction = model.predict([[usage, contract]])

if prediction[0] == 1:
    print("Customer may churn")
else:
    print("Customer may not churn")

```

Output:

The screenshot shows the OnlineGDB web interface. The code editor contains a Python script for logistic regression. The input values are 5 for usage minutes and 23 for contract duration. The output shows the prediction result.

```

1 import numpy as np
2 from sklearn.linear_model import LogisticRegression
3
4 # Features: usage minutes, contract months
5 X = np.array([
6     [100, 24], [120, 24], [80, 12],
7     [300, 6], [280, 6], [350, 3]
8 ])
9
10 # 0 = Not churned, 1 = Churned
11 y = np.array([0, 0, 0, 1, 1, 1])
12
13 model = LogisticRegression()
14 model.fit(X, y)
15
16 usage = float(input("Usage minutes: "))
17 contract = float(input("Contract duration: "))
18
19 prediction = model.predict([[usage, contract]])
20
21 if prediction[0] == 1:
22     print("Customer may churn")
23 else:
24     print("Customer may not churn")
25

```

Input:
Usage minutes: 5
Contract duration: 23
Customer may not churn

...Program finished with exit code 0
Press ENTER to exit console

Question 28: K-Means Customer Segmentation

Program code:

```

import numpy as np
from sklearn.cluster import KMeans

X = np.array([
    [1000, 2],
    [1200, 3],
    [1100, 2],
    [5000, 10],
    [5200, 11],
    [4800, 9]
])

model = KMeans(n_clusters=2, random_state=1, n_init=10)
model.fit(X)

```

```

new_customer = [[1500, 3]]
cluster = model.predict(new_customer)

print("Customer belongs to cluster:", cluster[0])

```

Output:

The screenshot shows the OnlineGDB web interface. The left sidebar contains navigation links: OnlineGDB, code.compile.run.debug.share., IDE, My Projects, Classroom (highlighted), Learn Programming, Programming Questions, Sign Up, and Login. The main editor area displays a Python script for KMeans clustering. The script imports numpy and KMeans from sklearn, defines a dataset X, fits a KMeans model with 2 clusters, and predicts the cluster for a new customer. The output console shows the result: 'Customer belongs to cluster: 0'. The bottom status bar indicates the program finished with exit code 0.

```

main.py
1 import numpy as np
2 from sklearn.cluster import KMeans
3
4 X = np.array([
5     [1000, 2],
6     [1200, 3],
7     [1100, 2],
8     [5000, 10],
9     [5200, 11],
10    [4800, 9]
11 ])
12
13 model = KMeans(n_clusters=2, random_state=1, n_init=10)
14 model.fit(X)
15
16 new_customer = [[1500, 3]]
17 cluster = model.predict(new_customer)
18
19 print("Customer belongs to cluster:", cluster[0])
20

```

input

Customer belongs to cluster: 0

...Program finished with exit code 0
Press ENTER to exit console

Question 29: Accuracy, Precision, Recall and F1-Score

Program code:

```

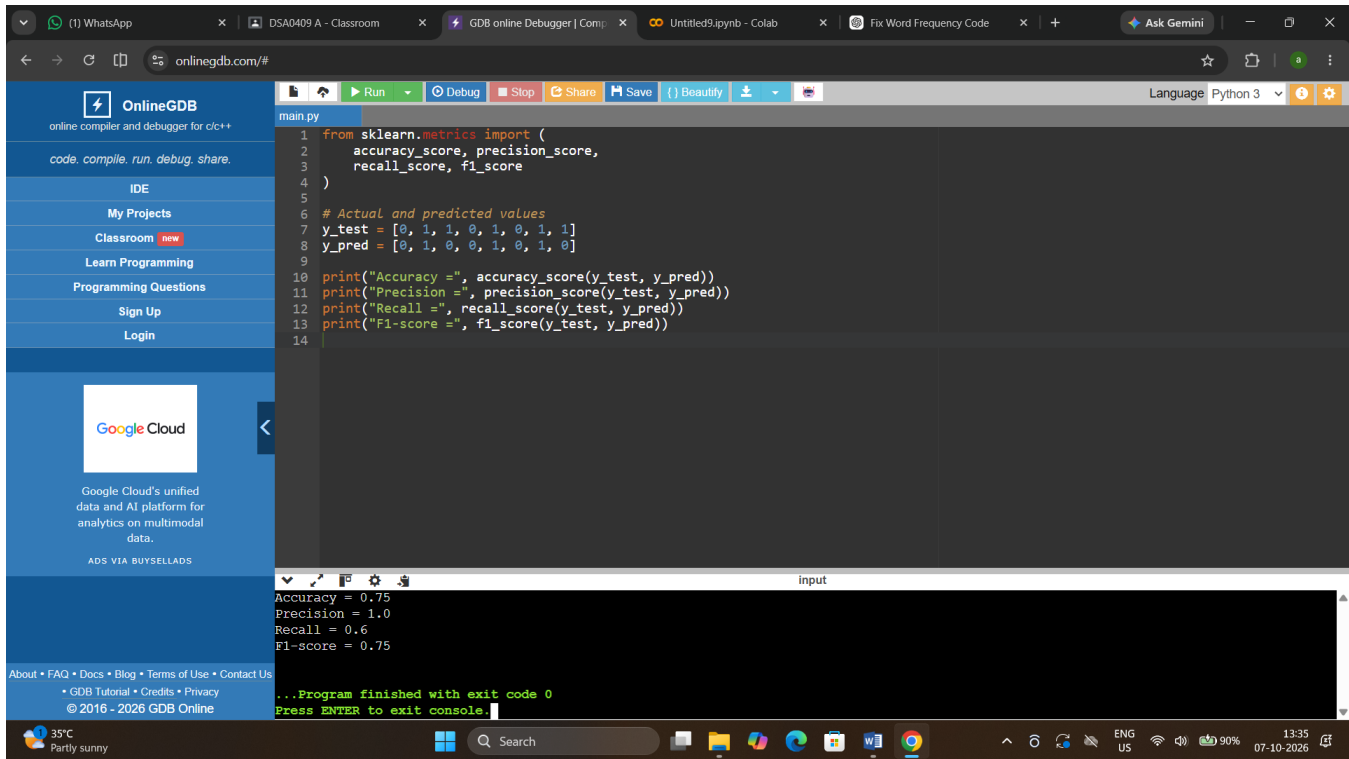
from sklearn.metrics import (
    accuracy_score, precision_score,
    recall_score, f1_score
)

# Actual and predicted values
y_test = [0, 1, 1, 0, 1, 0, 1, 1]
y_pred = [0, 1, 0, 0, 1, 0, 1, 0]

print("Accuracy =", accuracy_score(y_test, y_pred))
print("Precision =", precision_score(y_test, y_pred))
print("Recall =", recall_score(y_test, y_pred))
print("F1-score =", f1_score(y_test, y_pred))

```

Output:



The screenshot shows the OnlineGDB web interface. The left sidebar contains navigation links: OnlineGDB, code.compile.run.debug.share, IDE, My Projects, Classroom (highlighted), Learn Programming, Programming Questions, Sign Up, and Login. The main editor area displays a Python script in a dark-themed editor. The script imports metrics from sklearn, defines test and predicted values, and prints the accuracy, precision, recall, and F1-score. The output console at the bottom shows the results of the script execution.

```
main.py
1 from sklearn.metrics import (
2     accuracy_score, precision_score,
3     recall_score, f1_score
4 )
5
6 # Actual and predicted values
7 y_test = [0, 1, 1, 0, 1, 0, 1, 1]
8 y_pred = [0, 1, 0, 0, 1, 0, 1, 0]
9
10 print("Accuracy =", accuracy_score(y_test, y_pred))
11 print("Precision =", precision_score(y_test, y_pred))
12 print("Recall =", recall_score(y_test, y_pred))
13 print("F1-score =", f1_score(y_test, y_pred))
14
```

input

```
Accuracy = 0.75
Precision = 1.0
Recall = 0.6
F1-score = 0.75

...Program finished with exit code 0
Press ENTER to exit console
```

Question 30: CART Regression for Used Car Price

Program code:

```
import numpy as np
from sklearn.tree import DecisionTreeRegressor

# Features: mileage, age
X = np.array([
    [20000, 2], [30000, 3], [40000, 4],
    [60000, 5], [80000, 7], [100000, 9]
])

price = np.array([18000, 17000, 15000, 13000, 10000, 7000])

model = DecisionTreeRegressor(max_depth=3, random_state=1)
model.fit(X, price)
```

```

mileage = float(input("Mileage: "))
age = float(input("Age: "))

prediction = model.predict([[mileage, age]])

print("Predicted price =", prediction[0])

```

Output:

The screenshot shows the OnlineGDB web interface. The left sidebar contains navigation links like 'IDE', 'My Projects', 'Classroom', 'Learn Programming', 'Programming Questions', 'Sign Up', and 'Login'. The main editor area displays a Python script for a Decision Tree Regressor. The script imports numpy and sklearn, defines features (mileage, age), fits a model, and takes user input to make a prediction. The output console at the bottom shows the program execution with the input 'Mileage: 63' and 'Age: 19', resulting in a 'Predicted price = 7000.0'.

```

main.py
1 import numpy as np
2 from sklearn.tree import DecisionTreeRegressor
3
4 # Features: mileage, age
5 X = np.array([
6     [20000, 2], [30000, 3], [40000, 4],
7     [60000, 5], [80000, 7], [100000, 9]
8 ])
9
10 price = np.array([18000, 17000, 15000, 13000, 10000, 7000])
11
12 model = DecisionTreeRegressor(max_depth=3, random_state=1)
13 model.fit(X, price)
14
15 mileage = float(input("Mileage: "))
16 age = float(input("Age: "))
17
18 prediction = model.predict([[mileage, age]])
19
20 print("Predicted price =", prediction[0])
21

```

input

```

Mileage: 63
Age: 19
Predicted price = 7000.0
...Program finished with exit code 0
Press ENTER to exit console

```

Question 31: Customer Segmentation using K-Means

Program code:

```

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

data = pd.DataFrame({
    "Spending": [1000, 1200, 1100, 5000, 5200, 4800],
    "Visits": [2, 3, 2, 10, 11, 9]
})

model = KMeans(n_clusters=2, random_state=1, n_init=10)
data["Cluster"] = model.fit_predict(
    data[["Spending", "Visits"]]
)

```

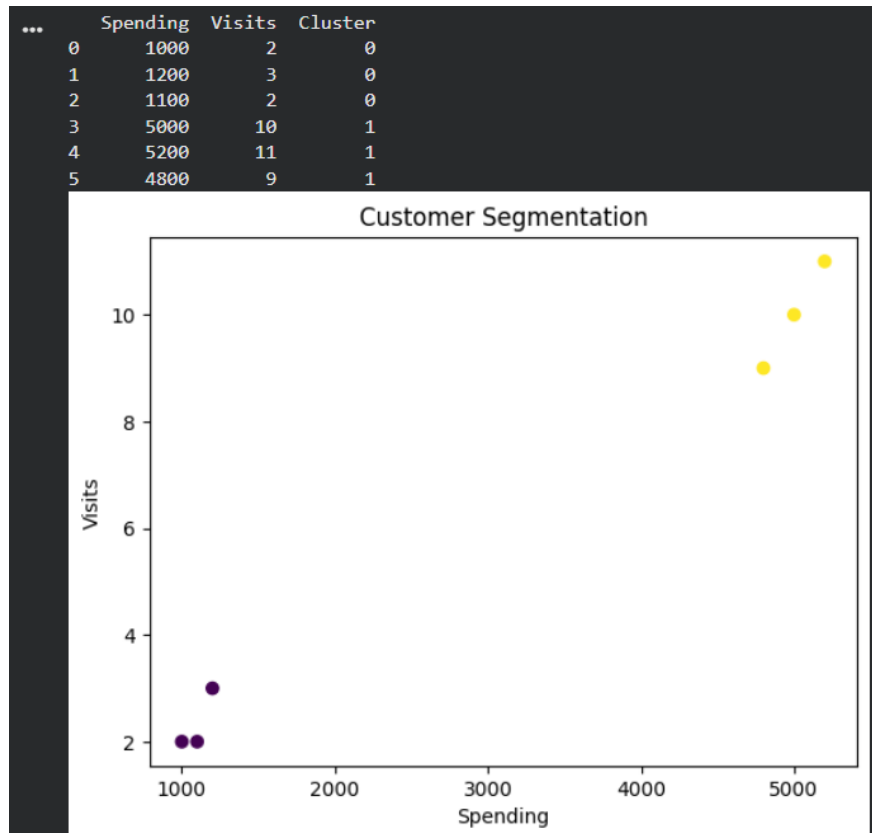
```

print(data)

plt.scatter(
    data["Spending"],
    data["Visits"],
    c=data["Cluster"]
)
plt.xlabel("Spending")
plt.ylabel("Visits")
plt.title("Customer Segmentation")
plt.show()

```

Output:



Question 32: Bivariate Analysis and Linear Regression for House Price

Program code:

```

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

data = pd.DataFrame({
    "Size": [800, 1000, 1200, 1500, 1800, 2000],
    "Price": [160, 200, 240, 300, 360, 400]
})

X = data[["Size"]]
y = data["Price"]

model = LinearRegression()

```

```

model.fit(X, y)

predicted = model.predict(X)

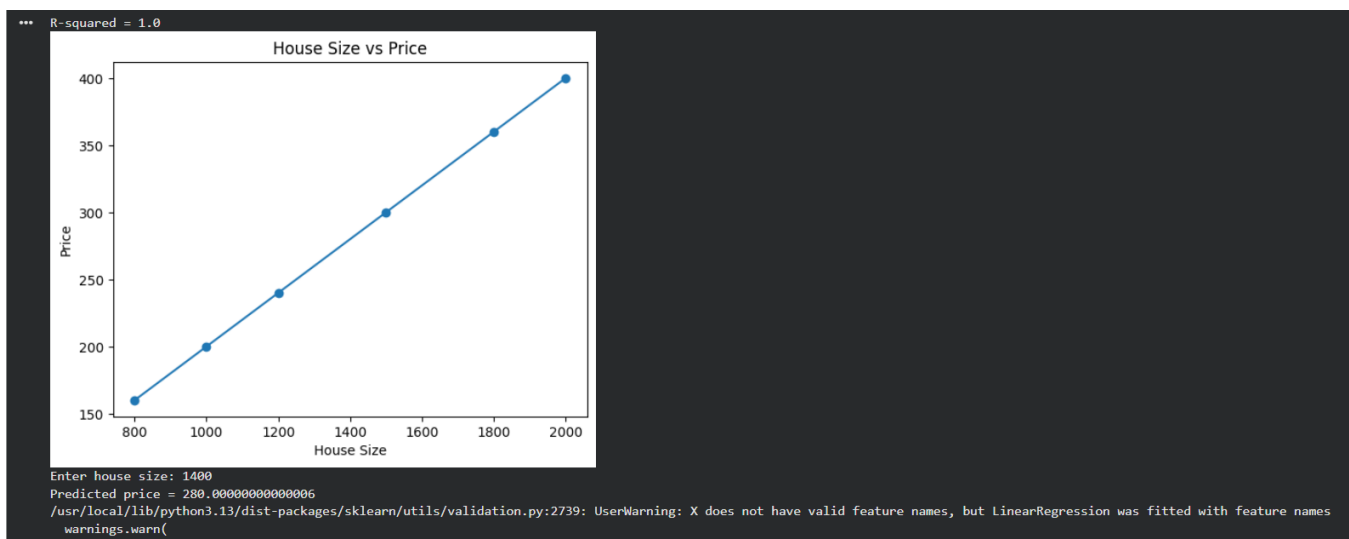
print("R-squared =", r2_score(y, predicted))

plt.scatter(data["Size"], data["Price"])
plt.plot(data["Size"], predicted)
plt.xlabel("House Size")
plt.ylabel("Price")
plt.title("House Size vs Price")
plt.show()

new_size = float(input("Enter house size: "))
print("Predicted price =", model.predict([[new_size]])[0])

```

Output:



Question 33: Multiple Linear Regression for Car Price

Program code:

```

import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

data = pd.DataFrame({
    "Engine": [1.2, 1.5, 1.8, 2.0, 2.2, 2.5],
    "Horsepower": [80, 100, 120, 140, 160, 190],
    "Mileage": [20, 18, 16, 15, 13, 11],
    "Price": [8000, 10000, 13000, 16000, 19000, 23000]
})

X = data[["Engine", "Horsepower", "Mileage"]]
y = data["Price"]

```

```

model = LinearRegression()
model.fit(X, y)

predicted = model.predict(X)

print("R-squared =", r2_score(y, predicted))
print("Coefficients =", model.coef_)

new_car = [[1.8, 120, 16]]
print("Predicted price =", model.predict(new_car)[0])

```

Output:

The screenshot shows the OnlineGDB web interface. The left sidebar contains navigation links like 'OnlineGDB', 'code.compile.run.debug.share.', 'IDE', 'My Projects', 'Classroom', 'Learn Programming', 'Programming Questions', 'Sign Up', and 'Login'. The main area displays a Python script for Linear Regression. The output console shows the following results:

```

R-squared = 0.9983049479376866
Coefficients = [-2692.30769231  180.76923077  115.38461538]
/nlx/store/8wki014rrpscmkwj5139fd6bsvg844zz-python3-3.13.7-env/lib/python3.13/site-packages/sklearn/utils/validation.py:2749: UserWarning
: X does not have valid feature names, but LinearRegression was fitted with feature names
warnings.warn(
Predicted price = 12961.538461538466
...Program finished with exit code 0
Press ENTER to exit console.

```

Question 34: KNN Classification with Evaluation Metrics

Program code:

```

import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score, precision_score,
    recall_score, f1_score
)

X = np.array([
    [20, 120], [22, 130], [25, 125], [27, 140],
    [40, 180], [42, 190], [45, 185], [48, 200]
])

```

```

y = np.array([0, 0, 0, 0, 1, 1, 1, 1])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=1
)

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy =", accuracy_score(y_test, y_pred))
print("Precision =", precision_score(y_test, y_pred))
print("Recall =", recall_score(y_test, y_pred))
print("F1-score =", f1_score(y_test, y_pred))

new_patient = [[30, 150]]
print("Prediction =", model.predict(new_patient))

```

Output:

```

main.py
1 import numpy as np
2 from sklearn.model_selection import train_test_split
3 from sklearn.neighbors import KNeighborsClassifier
4 from sklearn.metrics import (
5     accuracy_score, precision_score,
6     recall_score, f1_score
7 )
8
9 X = np.array([
10     [28, 128], [22, 138], [25, 125], [27, 148],
11     [48, 188], [42, 198], [45, 185], [48, 288]
12 ])
13
14 y = np.array([0, 0, 0, 0, 1, 1, 1, 1])
15
16 X_train, X_test, y_train, y_test = train_test_split(
17     X, y, test_size=0.25, random_state=1
18 )
19
20 model = KNeighborsClassifier(n_neighbors=3)
21 model.fit(X_train, y_train)
22
23 y_pred = model.predict(X_test)
24
25 print("Accuracy =", accuracy_score(y_test, y_pred))
26 print("Precision =", precision_score(y_test, y_pred))

```

Accuracy = 1.0
Precision = 1.0
Recall = 1.0
F1-score = 1.0
Prediction = 0

...Program finished with exit code 0
Press ENTER to exit console.

Question 35: K-Means Clustering for Retail Customers

Program code:

```

import pandas as pd
from sklearn.cluster import KMeans

data = pd.DataFrame({
    "Spending": [500, 700, 600, 4000, 4500, 4200, 1500, 1700],
    "Visits": [2, 3, 2, 10, 12, 11, 5, 6]
})

model = KMeans(n_clusters=3, random_state=1, n_init=10)

data["Cluster"] = model.fit_predict(
    data[["Spending", "Visits"]]
)

```

```

)

print(data)

print("Cluster centers:")
print(model.cluster_centers_)

```

Output:

The screenshot shows the OnlineGDB IDE interface. The code editor contains the following Python code:

```

1 import pandas as pd
2 from sklearn.cluster import KMeans
3
4 data = pd.DataFrame({
5     "Spending": [500, 700, 600, 4000, 4500, 4200, 1500, 1700],
6     "Visits": [2, 3, 2, 10, 12, 11, 5, 6]
7 })
8
9 model = KMeans(n_clusters=3, random_state=1, n_init=10)
10
11 data["Cluster"] = model.fit_predict(
12     data[["Spending", "Visits"]]
13 )
14
15 print(data)
16
17 print("Cluster centers:")
18 print(model.cluster_centers_)

```

The output console shows the following results:

```

input
Spending  Visits  Cluster
0         500     2        1
1         700     3        1
2         600     2        1
3        4000    10        0
4        4500    12        0
5        4200    11        0
6        1500     5        2
7        1700     6        2

Cluster centers:
[[4.23333333e+03 1.10000000e+01]
 [6.00000000e+02 2.33333333e+00]
 [1.60000000e+03 5.50000000e+00]]

...Program finished with exit code 0
Press ENTER to exit console

```

Question 36: Stock Price Variability from CSV

Program code:

```

import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_csv("stock_data.csv")

prices = data["Close"]

print("Mean =", prices.mean())
print("Standard deviation =", prices.std())
print("Minimum =", prices.min())
print("Maximum =", prices.max())
print("Range =", prices.max() - prices.min())

```

```
plt.plot(prices)
plt.title("Stock Closing Prices")
plt.xlabel("Trading Day")
plt.ylabel("Closing Price")
plt.show()
```

Output:

```
*** Mean = 113.13333333333334
Standard deviation = 7.763160868271806
Minimum = 100
Maximum = 125
Range = 25
```



Question 37: Correlation between Study Time and Exam Score

Program code:

```
import pandas as pd
import matplotlib.pyplot as plt

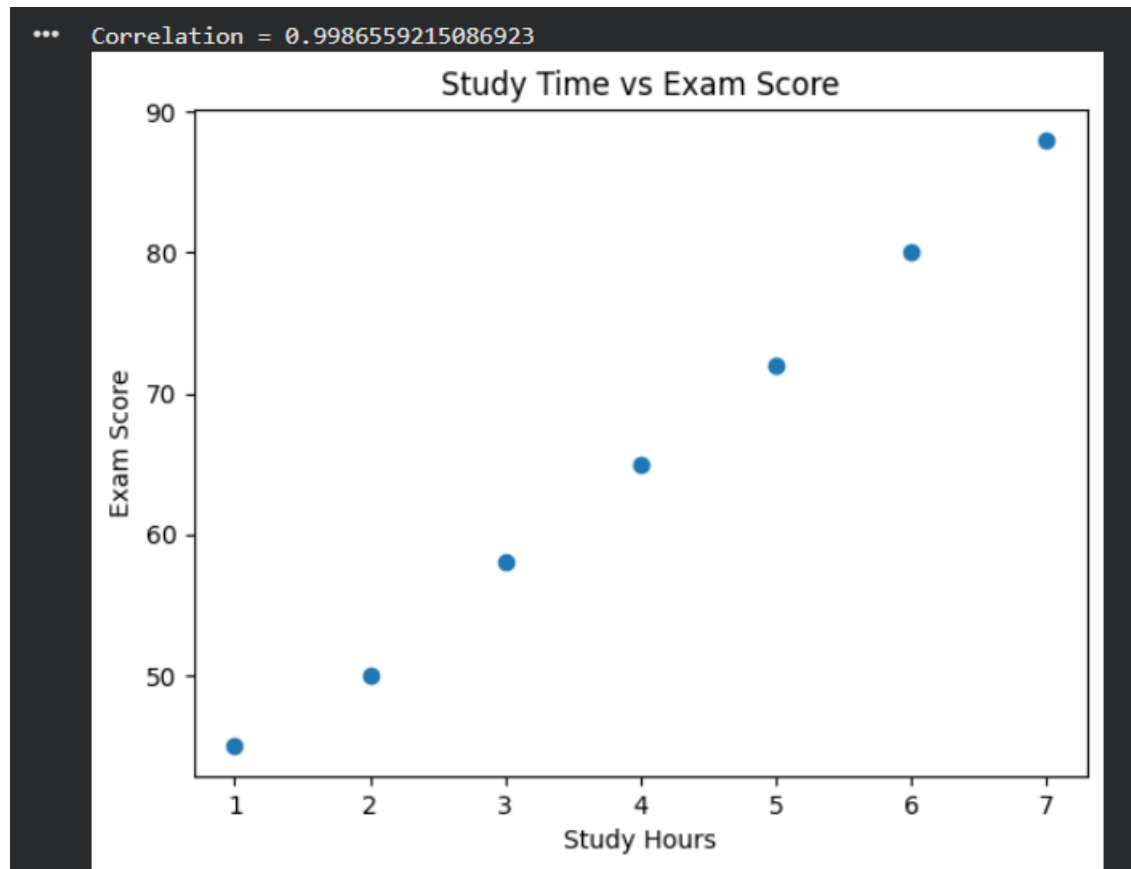
data = pd.DataFrame({
    "Study_Hours": [1, 2, 3, 4, 5, 6, 7],
    "Score": [45, 50, 58, 65, 72, 80, 88]
})

correlation = data["Study_Hours"].corr(data["Score"])

print("Correlation =", correlation)
```

```
plt.scatter(data["Study_Hours"], data["Score"])
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Study Time vs Exam Score")
plt.show()
```

Output:



Question 38: Temperature Analysis for Different Cities

Program code:

```
import numpy as np

# Rows = cities, columns = daily readings
temperatures = np.array([
    [25, 27, 30, 28, 26],
    [20, 21, 22, 21, 20],
    [30, 32, 35, 33, 31]
])

cities = ["Chennai", "Bengaluru", "Delhi"]
```

```

mean = np.mean(temperatures, axis=1)
std = np.std(temperatures, axis=1)
temperature_range = np.ptp(temperatures, axis=1)

for i in range(3):
    print(cities[i])
    print("Mean =", mean[i])
    print("Standard deviation =", std[i])
    print("Range =", temperature_range[i])

highest_range = np.argmax(temperature_range)
most_consistent = np.argmin(std)

print("Highest temperature range:", cities[highest_range])
print("Most consistent city:", cities[most_consistent])

```

Output:

The screenshot shows the OnlineGDB web interface. The code editor contains the following Python code:

```

1 import numpy as np
2
3 # Rows = cities, columns = daily readings
4 temperatures = np.array([
5     [25, 27, 30, 28, 26],
6     [20, 21, 22, 21, 20],
7     [30, 32, 35, 33, 31]
8 ])
9
10 cities = ["Chennai", "Bengaluru", "Delhi"]
11
12 mean = np.mean(temperatures, axis=1)
13 std = np.std(temperatures, axis=1)
14 temperature_range = np.ptp(temperatures, axis=1)
15
16 for i in range(3):
17     print(cities[i])
18     print("Mean =", mean[i])
19     print("Standard deviation =", std[i])
20     print("Range =", temperature_range[i])
21

```

The output of the program is displayed in the console:

```

Chennai
Mean = 27.2
Standard deviation = 1.7204650534085253
Range = 5
Bengaluru
Mean = 20.8
Standard deviation = 0.7483314773547882
Range = 2
Delhi
Mean = 32.2
Standard deviation = 1.7204650534085253
Range = 5
Highest temperature range: Chennai
Most consistent city: Bengaluru

```

Question 39: K-Means Clustering and Visualization

Program code:

```

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

data = pd.DataFrame({
    "Spending": [500, 600, 550, 3000, 3200, 3100, 1500, 1600],
    "Items": [2, 3, 2, 10, 11, 9, 5, 6]
})

model = KMeans(n_clusters=3, random_state=1, n_init=10)

```

```

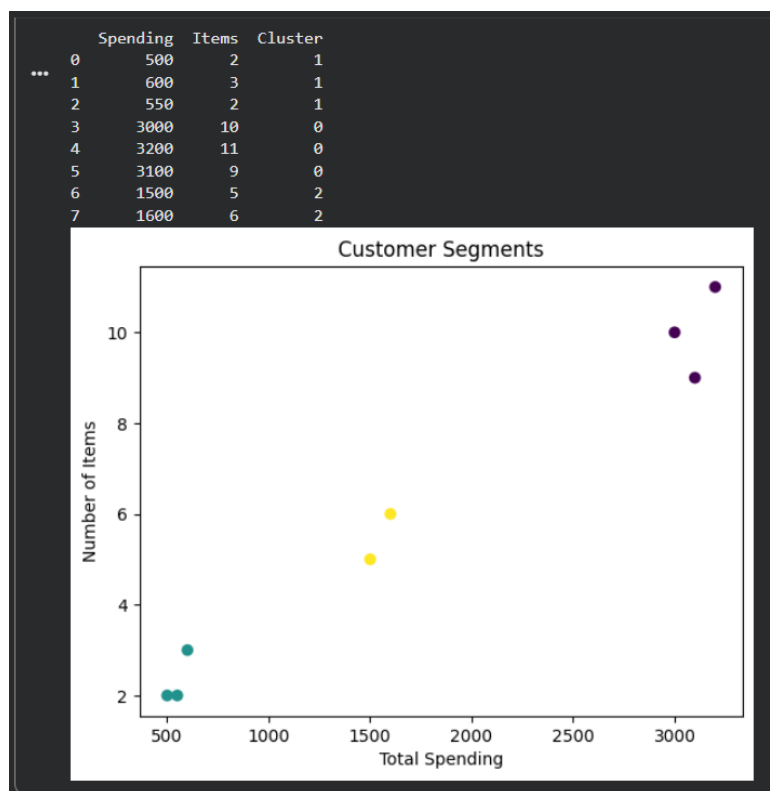
data["Cluster"] = model.fit_predict(
    data[["Spending", "Items"]]
)

print(data)

plt.scatter(
    data["Spending"],
    data["Items"],
    c=data["Cluster"]
)
plt.xlabel("Total Spending")
plt.ylabel("Number of Items")
plt.title("Customer Segments")
plt.show()

```

Output:



Question 40: Soccer Player Analysis using Pandas

Program code:

```

import pandas as pd
import matplotlib.pyplot as plt

# Create and save the dataset
data = pd.DataFrame({
    "Name": ["A", "B", "C", "D", "E", "F", "G"],
    "Age": [22, 25, 28, 24, 30, 27, 23],
    "Position": ["Forward", "Midfielder", "Forward",
                 "Defender", "Goalkeeper", "Forward", "Defender"],
    "Goals": [15, 8, 20, 4, 1, 12, 5],
    "Salary": [5000, 4500, 7000, 3500, 4000, 6000, 3800]
})

```

```

})

data.to_csv("soccer_players.csv", index=False)

# Read CSV
players = pd.read_csv("soccer_players.csv")

print("Top 5 goal scorers:")
print(players.nlargest(5, "Goals")[["Name", "Goals"]])

print("
Top 5 salaries:")
print(players.nlargest(5, "Salary")[["Name", "Salary"]])

average_age = players["Age"].mean()
print("
Average age =", average_age)

print("
Players above average age:")
print(players[players["Age"] > average_age][["Name"]])

players["Position"].value_counts().plot(kind="bar")
plt.title("Players by Position")
plt.xlabel("Position")
plt.ylabel("Number of Players")
plt.show()

```

Output:

